

Data Mining Notes

Academic Year 2026/2027

Contents

1	Introduction to Data Mining and the KDD Process	1
1.1	Foundations, Motivations, and Formal Definition	1
1.2	The Confluence of Disciplines and the KDD Process	3
1.3	Taxonomy of Data Mining Tasks	6
1.4	Open Challenges in Modern Data Mining	8
2	Data Understanding I: Data Types, Attributes, and Descriptive Statistics	10
2.1	Understanding Data and Dataset Taxonomy	10
2.2	Attribute Types and Measurement Scales	12
2.3	Data Quality Issues	14
2.4	Univariate Distribution Inspection	14
2.5	Measures of Central Tendency and Dispersion	16
2.6	Five-Number Summary, Quartiles, and Box Plots	17
3	Data Understanding II – Multivariate Analysis, Correlation, and Outlier Detection	19
3.1	Multidimensional Visualization Techniques	19
3.2	Bivariate Dependence Analysis: Covariance and Correlation .	24
3.3	Outlier Taxonomy and Detection Methodologies	28
3.4	Introduction to Missing Values	29
4	Data Preparation & Cleaning – Reduction, Discretization, and Transformation	31
4.1	Distinction Between Data Understanding and Data Preparation	31
4.2	Aggregation and Sampling	32
4.3	Dimensionality Reduction	34
4.4	Principal Component Analysis (PCA)	36
4.5	Handling Missing Values	39
4.6	Discretization and Binarization Techniques	40
4.7	Functional Attribute Transformations and Variance Stabilization	42

4.8	Normalization Algorithms	44
4.9	Univariate Outlier Treatment	45
5	Python Lab: Data Understanding and Data Preparation	48
5.1	Environment, Ingestion, and Data Integration	49
5.2	Missing Value Diagnosis and Imputation	51
5.3	Multivariate Exploration and Correlation Analysis	52
5.4	Feature Transformation, Discretization, and Encoding	54
5.5	Dimensionality Reduction: PCA and <i>t</i> -SNE	56
6	Proximity Measures, Information Theory, and Cluster Analysis	59
6.1	Foundations of Similarity, Dissimilarity, and Proximity	59
6.2	Single-Attribute Proximity Measures	60
6.3	Vector Distances and Minkowski Metrics	62
6.4	Worked Example of Distance Matrix Computation	63
6.5	Metric Axioms and Mathematical Properties	64
6.6	Binary Proximity Measures: SMC vs Jaccard Index	65
6.7	Cosine Similarity for Document Data	66
6.8	Weighting, Missing Values, and Correlation Across Objects	67
6.9	Information Theory: Entropy and Mutual Information	68
6.10	Foundations and Objectives of Cluster Analysis	70
6.11	Clustering Taxonomies and Grouping Structures	71
6.12	Cluster Types and Mathematical Formulations	73
6.13	Input Data Characteristics and Algorithm Families	74
7	Cluster Analysis: K-Means and Hierarchical Clustering Algorithms	75
7.1	Introduction to Partitional and Hierarchical Cluster Analysis	75
7.2	K-Means: Lloyd's Algorithm and Voronoi Geometry	76
7.3	SSE Objective Function and Proof of Convergence	80
7.4	Choice of Parameter K, Local Minima, and Initialization	82
7.5	Initialization Heuristics and Variants of K-Means	84
7.6	Structural Limitations of K-Means and Adaptation Techniques	86
7.7	Foundations of Hierarchical Clustering	88
7.8	Agglomerative Algorithm and Linkage Criteria	89
7.9	MST-Based Divisive Hierarchical Clustering	95
7.10	Complexity, Structural Limits, and Synoptic Comparison	96

Chapter 1

Introduction to Data Mining and the KDD Process

1.1 Foundations, Motivations, and Formal Definition

1.1.1 What is Data Mining?

Data Mining (knowledge extraction from data) is the application of efficient computational algorithms and techniques to the analysis of large-scale data collections, aiming to discover previously unknown, non-trivial, valid, and potentially actionable patterns, regularities, correlations, and structures.

Conceptually, Data Mining is neither simple database querying (as in SQL relational queries) nor basic descriptive statistical computation. Its core objective is uncovering **hidden knowledge**: latent dependency structures, implicit models, and predictive relationships that remain undetected through manual inspection or conventional analytical tools.

Definition 1.1.1 (Data Mining). **Data Mining** is the computational process of automatically or semi-automatically discovering patterns, regularities, correlations, or latent structures within massive datasets (*Big Data*), yielding valid, novel, useful, and interpretable models for human decision-makers.

1.1.2 Application Context: Commercial and Scientific Drivers

The necessity of computational disciplines dedicated to pattern extraction stems from the exponential expansion of data generated, stored, and processed globally:

- **Commercial Perspective:** E-commerce transactions, web browsing logs, retail scanner logs, and financial flows (credit card transactions, wire transfers, POS logs) generate continuous data streams in the order of terabytes or petabytes daily. Modern enterprises and financial institutions cannot gain strategic leverage through passive archival; they require transforming raw logs into behavioral insights for Customer Relationship Management (CRM), personalized recommendations, churn prevention, and real-time fraud mitigation.
- **Scientific Perspective:** Remote sensing and advanced instrumentation produce unprecedented data volumes. Earth observation satellites (e.g., the NASA EOSDIS program) ingest petabytes of environmental measurements yearly; digital sky surveys scan billions of celestial objects; high-throughput genomics generates nucleotide sequences continuously; and computational physical and climate simulations yield terabytes within hours. Scientific hypothesis formulation and empirical analysis inherently require scalable automated synthesis.

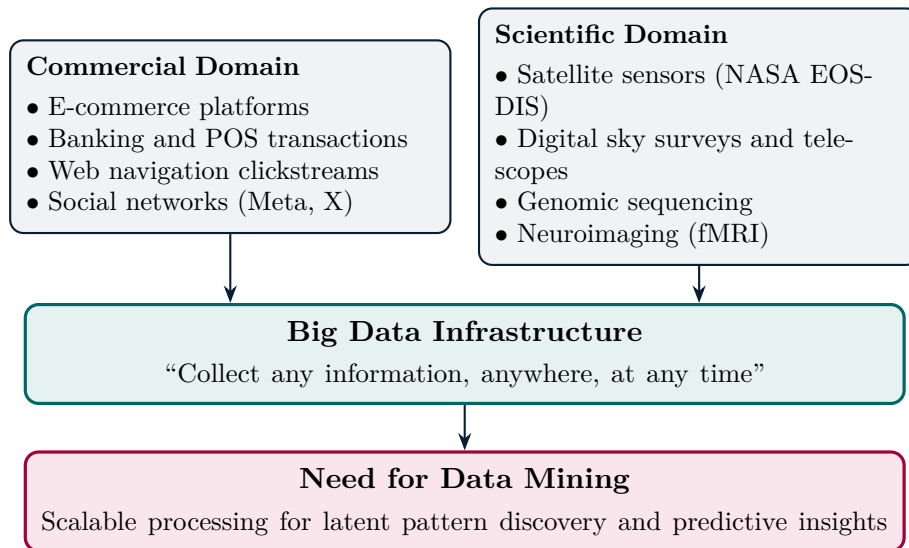


Figure 1.1: Genesis and evolution of Data Mining across commercial and scientific drivers.

1.1.3 Big Data as a Proxy for Human Activity and Global Challenges

Digital traces generated by connected devices, mobile nodes, and online services act as an empirical *quantitative proxy* for human behaviors and social

dynamics:

1. **Purchasing Patterns and Lifestyles:** Supermarket receipts, loyalty cards, and online baskets trace dietary habits, price elasticity, and household economics.
2. **Social Networks and Interactions:** Interaction graphs across social platforms reveal interpersonal contact topology and information diffusion dynamics.
3. **Public Sentiment and Collective Mood:** Search queries, microblogging posts, and collaborative encyclopedias measure public interests and sentiment trends in real time.
4. **Mobility and Transportation:** GPS sensors, cellular handovers, and air traffic matrices model mobility at both local levels (urban congestion) and global scales (pandemic pathogen dispersal tracking via platforms such as *BioDiaspora*).

These data-driven techniques enable society to tackle critical global challenges: healthcare efficiency, smart-grid energy optimization, climate change impact forecasting, and agricultural yield planning.

1.2 The Confluence of Disciplines and the KDD Process

1.2.1 A Multidisciplinary Confluence

Data Mining is inherently cross-disciplinary, emerging at the intersection of several core fields:

- **Database Systems:** Provide the infrastructure for scalable data storage, indexing, secondary-memory access, and distributed retrieval.
- **Statistics:** Furnishes mathematical foundations for uncertainty quantification, sampling, hypothesis testing, and probabilistic modeling.
- **Machine Learning:** Introduces adaptive architectures and optimization algorithms capable of learning empirical rules to generalize on unseen data.

- **Algorithms and Computational Theory:** Ensures analytical procedures remain computationally feasible with respect to time and memory complexities.
- **Visualization:** Enables intuitive perceptual data exploration and the effective communication of analytical findings.

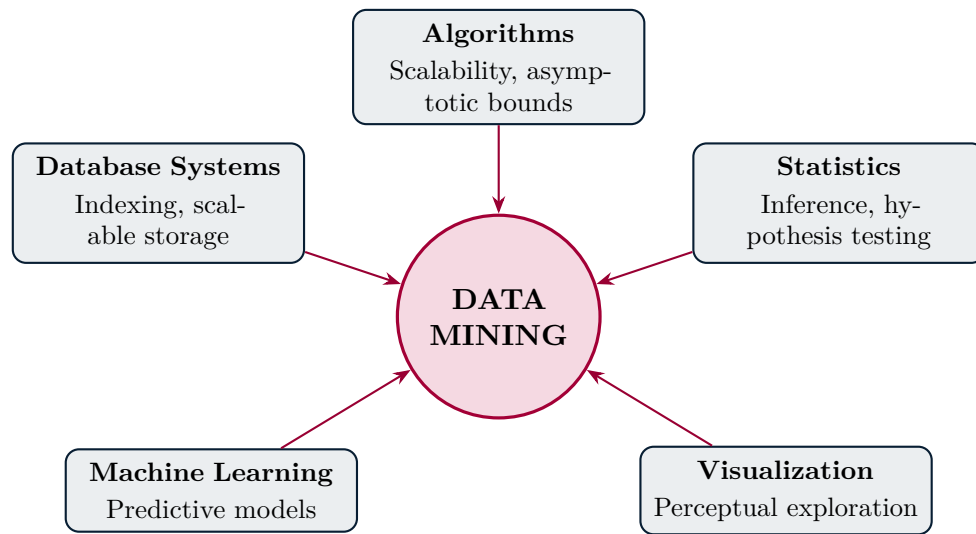


Figure 1.2: The multidisciplinary confluence of Data Mining across computational disciplines.

1.2.2 The KDD Process (Knowledge Discovery in Databases)

Data Mining is the algorithmic centerpiece of an iterative, multi-stage engineering methodology known as **KDD** (*Knowledge Discovery in Databases*):

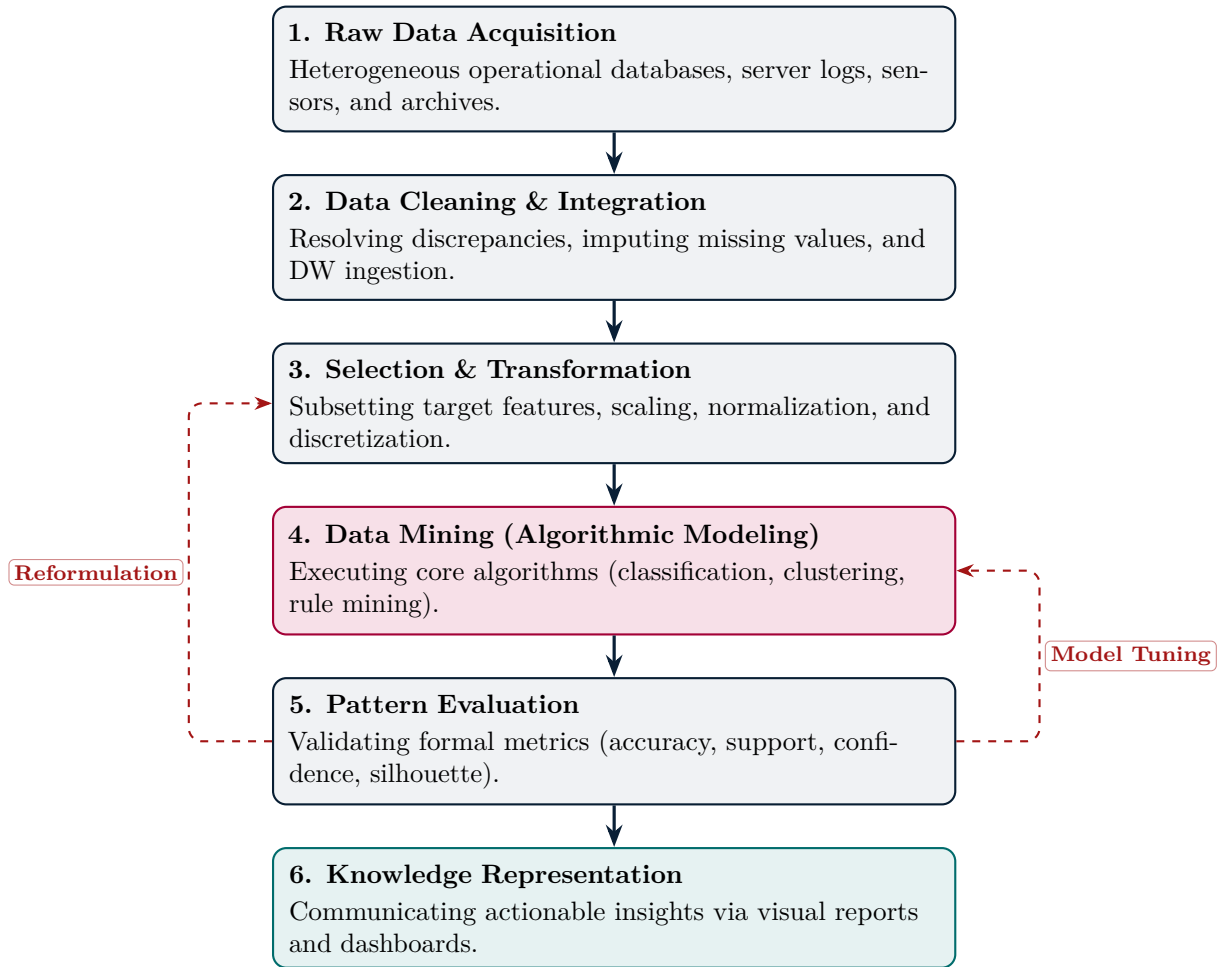


Figure 1.3: Step-by-step workflow of the KDD process with iterative feedback refinement loops.

Detailed Phases of the KDD Process:

1. **Business / Goal Understanding:** Formalizing domain objectives, defining KPIs, and setting operational constraints.
2. **Data Cleaning & Integration:** Reconciling fragmented data feeds, correcting noise and errors, and centralizing into a unified schema.
3. **Selection & Transformation:** Filtering target-specific entities and constructing engineered features.
4. **Data Mining:** Deploying supervised or unsupervised algorithms to uncover structural invariants.

5. **Pattern Evaluation:** Validating hypotheses against statistical benchmarks and domain expectations.
6. **Knowledge Representation:** Visualizing and operationalizing findings for stakeholder decision-making.

1.3 Taxonomy of Data Mining Tasks

Data Mining tasks are systematically categorized into two macro-classes:

1. **Predictive Methods:** Infer an inductive model mapping input predictors to an unknown continuous or categorical target attribute.
2. **Descriptive Methods:** Explore underlying groupings, correlations, and topological structures without an explicit target variable.

1.3.1 Classification (Predictive Modeling)

Classification maps an input vector $x \in \mathcal{X}$ to a discrete category label $y \in \mathcal{Y} = \{c_1, c_2, \dots, c_k\}$. Given a labeled training set:

$$\mathcal{D}_{\text{train}} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\} \quad (1.1)$$

the induction algorithm estimates an optimal classification mapping:

$$f : \mathcal{X} \rightarrow \mathcal{Y} \quad (1.2)$$

that minimizes expected classification error on unseen test samples.

Example: Credit Worthiness Assessment Consider a toy loan-granting dataset:

TID	Employed	Level of Education	Years at Address	Credit Worthy (Y)
1	Yes	Graduate	5	Yes
2	Yes	High School	2	No
3	No	Undergrad	1	No
4	Yes	High School	10	Yes

Table 1.1: Toy dataset for inductive decision tree credit evaluation.

A decision tree induction algorithm induces the following hierarchical structure:

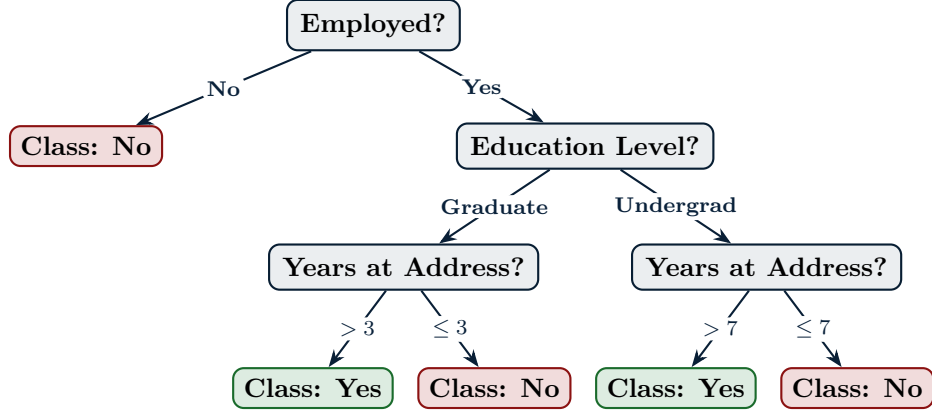


Figure 1.4: Decision tree inferred from the credit worthiness dataset.

1.3.2 Clustering (Unsupervised Partitioning)

Clustering partitions an unlabeled dataset $\mathcal{D} = \{x_1, x_2, \dots, x_N\}$ into k disjoint subsets $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ such that:

1. **Minimized Intra-Cluster Distance:** Items grouped within the same cluster C_i exhibit high mutual similarity.
2. **Maximized Inter-Cluster Distance:** Items in distinct clusters C_i and C_j ($i \neq j$) are well-separated in feature space.



Figure 1.5: Conceptual clustering objective: intra-group cohesion and inter-group separation.

1.3.3 Association Rule Discovery

Association analysis identifies co-occurrence affinities within transactional databases. Given an itemset $\mathcal{I} = \{i_1, i_2, \dots, i_m\}$ and a collection of transactions $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ with $t_j \subseteq \mathcal{I}$, an **association rule** is defined as:

$$X \implies Y \quad \text{where } X, Y \subset \mathcal{I} \text{ and } X \cap Y = \emptyset \quad (1.3)$$

indicating that transactions containing X (antecedent) systematically contain Y (consequent).

TID	Items Purchased
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diapers, Milk
4	Beer, Bread, Diapers, Milk
5	Coke, Diapers, Milk

Table 1.2: Market Basket transaction log.

Prominent extracted rules include:

$$\{\text{Milk}\} \implies \{\text{Coke}\}, \quad \{\text{Diapers, Milk}\} \implies \{\text{Beer}\} \quad (1.4)$$

1.3.4 Anomaly Detection (Deviation Detection)

Anomaly detection identifies rare samples or trajectories deviating substantially from standard data generation mechanisms, applied across network intrusion monitoring, payment fraud prevention, predictive equipment failure analysis, and ecological disturbance tracking.

1.4 Open Challenges in Modern Data Mining

1. **Scalability:** Sub-quadratic computational requirements to cope with petabyte-scale datasets.
2. **High Dimensionality:** Combating geometric volume inflation and distance concentration.
3. **Complex and Heterogeneous Data:** Harmonizing relational tables, graphs, textual corpora, temporal series, and high-frequency sensor streams.

4. **Distributed Ownership and Privacy:** Complying with regulatory frameworks and privacy constraints across distributed networks.
5. **Dynamic and Non-Stationary Environments:** Accommodating concept drift and online stream learning.

Chapter 2

Data Understanding I: Data Types, Attributes, and Descriptive Statistics

2.1 Understanding Data and Dataset Taxonomy

Data Understanding is the diagnostic investigation in which the analyst examines data quality, understands domain feature semantics, and uncovers initial indications of noise, extreme outliers, or missing values.

Core Objectives of Data Understanding

- What types of attributes constitute the dataset?
- What is the overall data quality, and what syntactic/semantic errors exist?
- Are there univariate or multivariate outliers present?
- What pairwise correlations and dependencies connect features?
- How are missing values distributed, and what mechanism generated them?
- Is feature transformation or feature engineering required prior to mining?

2.1.1 Fundamental Structure: Objects and Attributes

A dataset is a formal collection of **data objects** (also termed *records, points, samples, entities, instances, or cases*) described by a set of **attributes** (known as *variables, features, fields, or dimensions*).

- An **object** represents a single real-world entity (a customer, a bank transaction, a sensor reading).
- An **attribute** denotes a measurable property or characteristic of that entity (height, price, temperature, category).

2.1.2 Classification of Data Structures

Datasets differ fundamentally by their underlying structural topology:

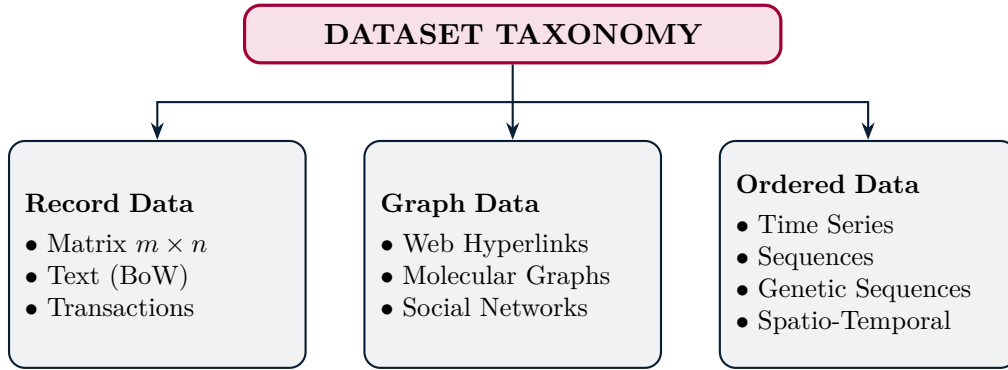


Figure 2.1: Structural taxonomy of major dataset formats.

1. Record Data

- **Data Matrix:** When all m objects share the same fixed set of n continuous numerical attributes, the dataset is modeled as an $m \times n$ matrix:

$$D = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n} \quad (2.1)$$

where each row i is a point in continuous Euclidean space $\mathbb{R}^n$, and each column is a coordinate axis.

- **Document Data (Document-Term Matrix):** Unstructured texts modeled via *Bag-of-Words*. Columns represent vocabulary terms and values record word frequencies:

Document	team	coach	play	ball	score	game	win	lost	timeout	season
Doc 1	3	0	5	0	2	6	0	2	0	2
Doc 2	0	7	0	2	1	0	0	3	0	0
Doc 3	0	1	0	0	1	2	2	0	3	0

Table 2.1: Document-Term Matrix representation for text collections.

- **Transaction Data:** Each record is identified by a unique TID and contains a variable-length set of discrete items (e.g., supermarket market basket).

2. Graph Data

Explicitly models pairwise entities: World Wide Web hyperlinks, molecular structures (e.g., benzene C_6H_6 covalent bonds), and citation networks.

3. Ordered Data

Data where temporal or spatial precedence governs informational semantics:

- **Time Series:** Continuous or discrete time-indexed readings with high autocorrelation.
- **Sequential Data:** Chronologically ordered event sequences (e.g., $(A, B) \rightarrow (D) \rightarrow (C, E)$).
- **Genomic Sequences:** Nucleotide polymer chains ('A', 'C', 'G', 'T').
- **Spatio-Temporal Data:** Space-time indexed matrices (e.g., global sea-surface temperature grids).

2.2 Attribute Types and Measurement Scales

Mathematical admissibility of operations on attribute domains:

- **Distinctness (= or $\neq$):** Testing equality between values.
- **Order (< or >):** Natural total or partial ordering.

- **Difference (+ or −):** Constant-metric measurable intervals.
- **Ratio (* or /):** Natural absolute zero (total absence of measured property).

Scale	Distinct.	Order	Diff.	Ratio	Permissible Statistical Operators
Nominal	✓	–	–	–	Mode, entropy, χ^2 test
Ordinal	✓	✓	–	–	Median, quantiles, Spearman rank ρ
Interval	✓	✓	✓	–	Mean, standard deviation, Pearson r
Ratio	✓	✓	✓	✓	Geometric mean, coefficient of variation

Table 2.2: Hierarchy and mathematical properties of measurement scales.

2.2.1 Categorical (Qualitative) Attributes

1. **Nominal:** Allows only equality comparison ($=, \neq$). Examples: eye color, zip code, tax ID.
2. **Binary:** Nominal attributes with exactly two values $\{0, 1\}$.
 - *Symmetric Binary:* Both outcomes carry equal weight (e.g., gender Male/Female).
 - *Asymmetric Binary:* One outcome is rare or of focal interest (1 = positive medical test, 0 = negative).
3. **Ordinal:** Natural ordering ($<, >$), but step distance is non-constant. Examples: Mohs hardness scale, ratings {poor, fair, good}, letter grades $\{A, B, C, D, F\}$.

2.2.2 Numeric (Quantitative) Attributes

1. **Interval-Scaled:** Differences are meaningful, but the scale **lacks an absolute zero**. Examples: Celsius or Fahrenheit temperature (20°C is not “twice as hot” as 10°C , since in Kelvin $293.15\text{ K} \neq 2 \cdot 283.15\text{ K}$); calendar dates.

2. **Ratio-Scaled:** Possesses a **natural absolute zero**. Examples: length, weight, age, Kelvin temperature (0 K absolute zero), monetary counts. Stating that a 3-hour match is 50% longer than a 2-hour match ($3/2 = 1.5$) is mathematically valid.

2.2.3 Discrete vs Continuous Variables

- **Discrete Attribute:** Finite or countably infinite set of values, represented by integers.
- **Continuous Attribute:** Real-valued domain ($\mathbb{R}$), represented by floating-point numbers.

2.3 Data Quality Issues

Flawed data invalidates downstream analytical pipelines (*Garbage In, Garbage Out*). Redman (2004) estimated poor data quality costs organizations 10% ÷ 20% of total revenue.

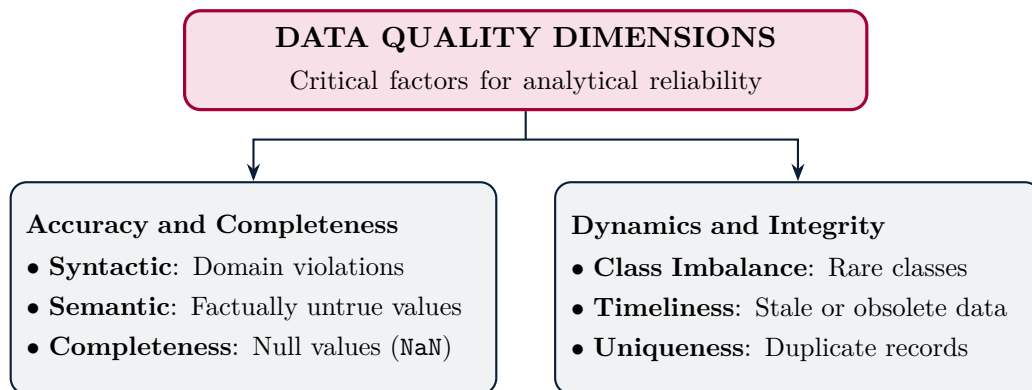


Figure 2.2: Core dimensions of data quality.

2.4 Univariate Distribution Inspection

2.4.1 Skewness and Multimodality

The shape of continuous probability density functions reveals underlying mechanisms:

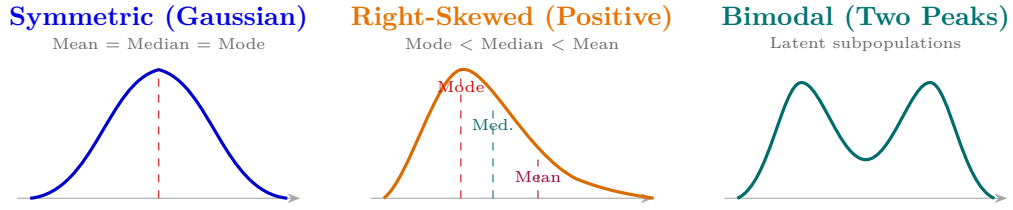


Figure 2.3: Univariate distribution profiles: symmetry, positive skewness, and multimodality.

- **Symmetric Distribution:** Reflectional symmetry about the center: Mean = Median = Mode.
- **Positively Skewed (Right-Skewed):** Long tail towards higher values (e.g., individual wealth):

$$\text{Mode} < \text{Median} < \text{Mean} \quad (2.2)$$

- **Negatively Skewed (Left-Skewed):** Tail stretching towards small values:

$$\text{Mean} < \text{Median} < \text{Mode} \quad (2.3)$$

- **Bimodal / Multimodal Distribution:** Signals overlapping subpopulations (e.g., adult height without conditioning on biological sex).

2.4.2 Visual Tools: Bar Charts vs Histograms

- **Bar Chart:** Plots discrete categories with visible spacing between bars.
- **Histogram:** Partitions continuous domains into contiguous intervals (*bins*); rectangle area is proportional to empirical sample counts.

Bin Sizing and Sturges' Rule: Under approximately normal distributions, Sturges' rule determines optimal bin counts k :

$$k = \lceil \log_2(n) + 1 \rceil \quad (2.4)$$

2.5 Measures of Central Tendency and Dispersion

2.5.1 Central Tendency

- **Sample Mean:**

$$\bar{x} = \frac{1}{m} \sum_{i=1}^m x_i \quad (2.5)$$

Highly vulnerable to severe outliers.

- **Median:** Positional center in sorted arrays:

$$\text{median} = \begin{cases} x_{(\frac{m+1}{2})} & \text{if } m \text{ is odd} \\ \frac{x_{(\frac{m}{2})} + x_{(\frac{m}{2}+1)}}{2} & \text{if } m \text{ is even} \end{cases} \quad (2.6)$$

A robust indicator resistant to extreme values.

- **Mode:** Most frequently observed value; uniquely defined across nominal attributes.
- **Trimmed Mean:** Evaluated by dropping a proportion α of extremes.

2.5.2 Measures of Dispersion

- **Range:** $\text{Range} = x_{\max} - x_{\min}$.
- **Sample Variance (s^2):**

$$s^2 = \frac{1}{m-1} \sum_{i=1}^m (x_i - \bar{x})^2 \quad (2.7)$$

using Bessel's correction ($m-1$) for unbiased population estimation.

- **Sample Standard Deviation (s):** $s = \sqrt{s^2}$.

Robust Dispersion Indicators:

- **Absolute Average Deviation (AAD):**

$$\text{AAD}(x) = \frac{1}{m} \sum_{i=1}^m |x_i - \bar{x}| \quad (2.8)$$

- **Median Absolute Deviation (MAD):**

$$\text{MAD}(x) = \text{median} \left(\left\{ |x_i - \text{median}(x)| \right\}_{i=1}^m \right) \quad (2.9)$$

2.6 Five-Number Summary, Quartiles, and Box Plots

2.6.1 The Five-Number Summary

The **Five-Number Summary** characterizes distributions using positional order statistics:

$$\{x_{\min}, Q_1, \text{Median } (Q_2), Q_3, x_{\max}\} \quad (2.10)$$

- Q_1 (First Quartile): 25th percentile.
- Q_2 (Median): 50th percentile.
- Q_3 (Third Quartile): 75th percentile.
- **Interquartile Range (IQR)**: $\text{IQR} = Q_3 - Q_1$.

2.6.2 Box Plot Anatomy

The **Box Plot** maps the five-number summary geometrically:

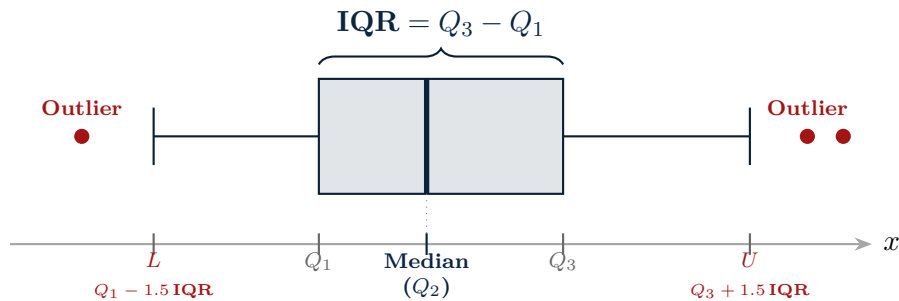


Figure 2.4: Box Plot anatomy with Tukey fences for univariate outlier demarcation.

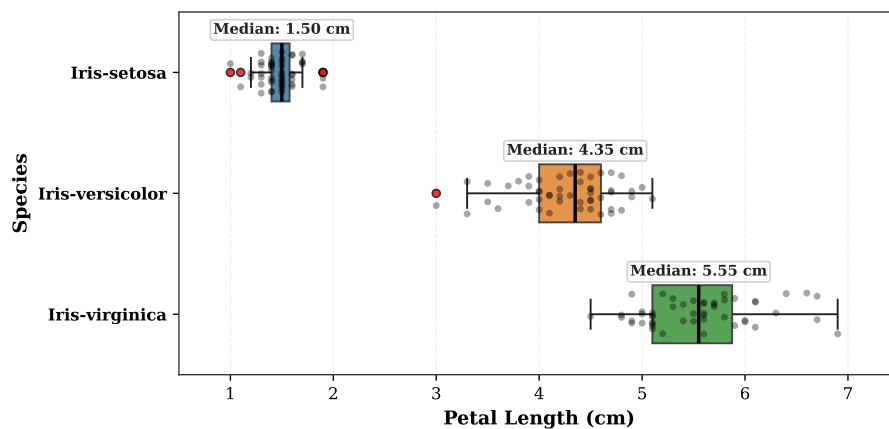


Figure 2.5: Empirical conditional box plots on the Iris dataset: petal length distributions partitioned by species.

Limitations of Box Plots Relative to Histograms: Box plots cannot detect multimodality. Identical five-number summaries can emerge from both bimodal and unimodal distributions; histograms remain necessary to reveal intermediate concavities.

Chapter 3

Data Understanding II – Multivariate Analysis, Correlation, and Outlier Detection

Multivariate analysis studies simultaneous relationships among two or more attributes within a dataset, overcoming the limitations of isolated univariate descriptions. In this chapter, we explore multidimensional visualization techniques, formal indices for linear and non-linear association and correlation, methodological criteria for outlier diagnosis, and an operational diagnostic checklist to conclude the Data Understanding phase.

The case study used as our benchmark is the famous **Iris** dataset (introduced by botanist Edgar Anderson and popularized by Ronald Fisher in 1936), composed of 150 observations evenly distributed across 3 balanced classes of 50 samples each (*Iris setosa*, *Iris versicolor*, *Iris virginica*) and described by 4 continuous features measured in centimeters:

- Sepal length (*Sepal Length*)
- Sepal width (*Sepal Width*)
- Petal length (*Petal Length*)
- Petal width (*Petal Width*)

3.1 Multidimensional Visualization Techniques

When exploratory analysis extends to bivariate and multivariate configurations, graphical representations must provide immediate visual insight

into geometric class separability, natural cluster formations, and structural anomalies.

3.1.1 Scatter Plot and Scatter Matrix

The **Scatter Plot** pairs two continuous attributes (X, Y) , projecting observations as points with Cartesian coordinates $(x_i, y_i) \in \mathbb{R}^2$.

- **Key Capabilities:** Visually highlights clusters, direct or inverse correlation trends, and bivariate outliers that remain undetectable in univariate histograms.
- **Color Encoding:** By assigning distinct colors or markers according to class labels, practitioners can assess the geometric separability induced by the feature pair.
- **Evidence from Iris:** Crossing *Petal Length* and *Petal Width* reveals sharp class separation: *Setosa* forms an isolated compact cluster, whereas *Versicolor* and *Virginica* exhibit only modest boundary overlap (Figure 3.1). In contrast, the plane formed by *Sepal Length* and *Sepal Width* shows substantial inter-class confusion between *Versicolor* and *Virginica*.

When the number of numeric features p is moderate ($3 \leq p \leq 10$), a **Scatter Matrix (Scatter Plot Matrix - SPLOM)** is employed: a symmetric $p \times p$ grid containing all $\frac{p(p-1)}{2}$ distinct pairwise scatter plots. Univariate histograms or kernel density estimates (KDE) are typically placed along the main diagonal, enabling simultaneous inspection of univariate distributions and pairwise correlations (Figure 3.2).

The **Three-Dimensional (3D) Scatter Plot** extends Cartesian projection to three coordinates simultaneously (X_1, X_2, X_3) . It allows identifying 3D outliers that appear completely normal in 2D projections, though planar displays suffer from visual occlusion and viewpoint dependence.

3.1.2 Visualizing High-Dimensional Spaces

In spaces where dimension count exceeds $p > 3$, standard orthogonal Cartesian projections fail to provide comprehensive views. Alternative coordinate geometries are required.

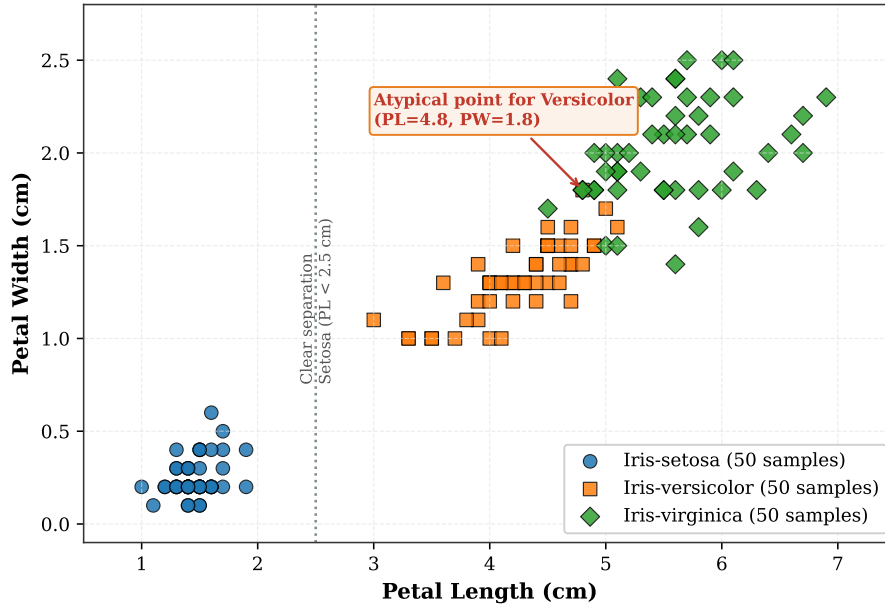


Figure 3.1: Bivariate scatter plots on the Iris dataset: comparing sepal and petal class separability.

Parallel Coordinates

In parallel coordinates, coordinate axes are not mutually orthogonal; instead, they are drawn as p equidistant, parallel vertical lines, each calibrated to the value range of its corresponding feature.

- **Geometric Mapping:** A p -dimensional observation $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip}) \in \mathbb{R}^p$ is represented not as a point, but as a continuous **polyline** crossing all parallel axes, intersecting the j -th vertical axis at height x_{ij} .
- **Diagnostic Traits:** Records sharing identical class labels tend to form compact bundles of polylines displaying coherent trajectories. An isolated polyline with an anomalous trajectory immediately flags a potential multivariate outlier.
- **Axis Ordering:** Visual legibility depends strongly on horizontal axis ordering. Placing strongly correlated attributes on adjacent axes reveals parallel bundles (positive correlation) or pencil-shaped X-crossings (negative correlation) (Figure 3.3).

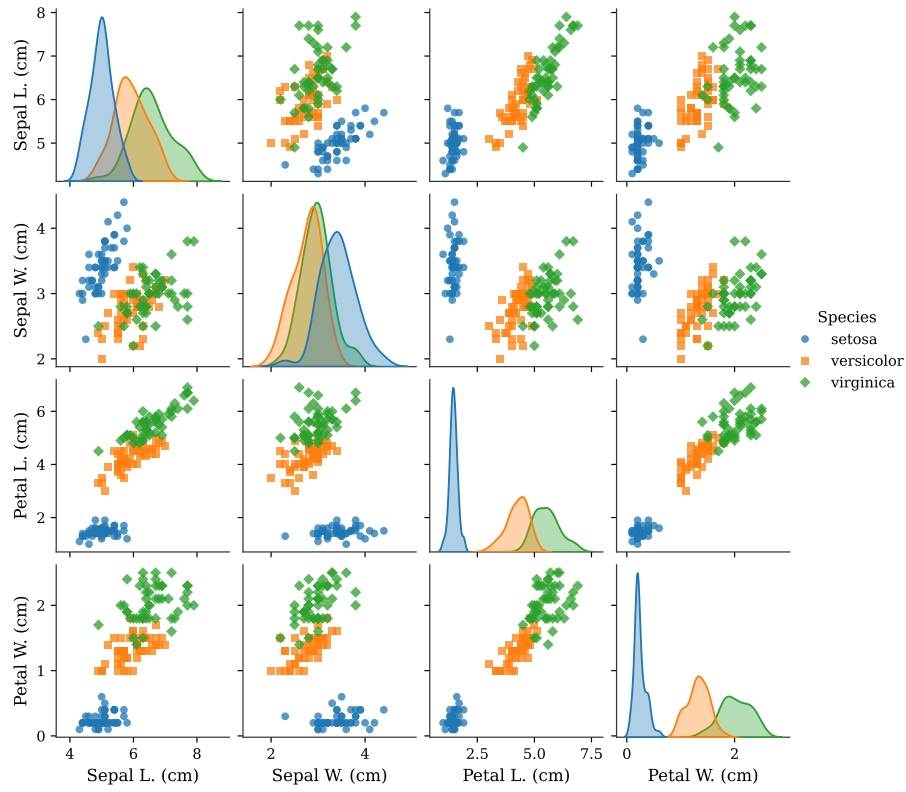


Figure 3.2: Complete Scatter Plot Matrix (SPLOM) across the four attributes of the Iris dataset.

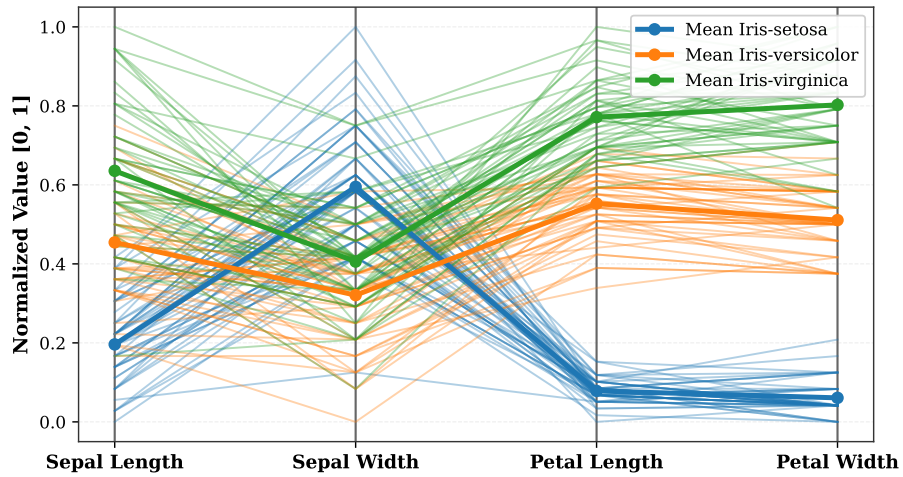


Figure 3.3: Parallel coordinates for the Iris dataset with class-stratified trajectories.

Radar Plot (Spider or Star Chart)

The **Radar Plot** shares the structural principle of parallel coordinates, but dimensional axes are arranged **radially emanating from a shared central pole**, separated by equi-spaced angles:

$$\theta_j = \frac{2\pi(j-1)}{p}, \quad j = 1, \dots, p \quad (3.1)$$

For each data record, feature values determine the vertex distance along the corresponding radial spoke; connecting consecutive vertices produces a closed polygon. When radar plots are displayed as small individual multiples for each observation across a grid, they are called **Star Plots**, facilitating rapid geometric comparisons across multivariate instances (Figure 3.4).

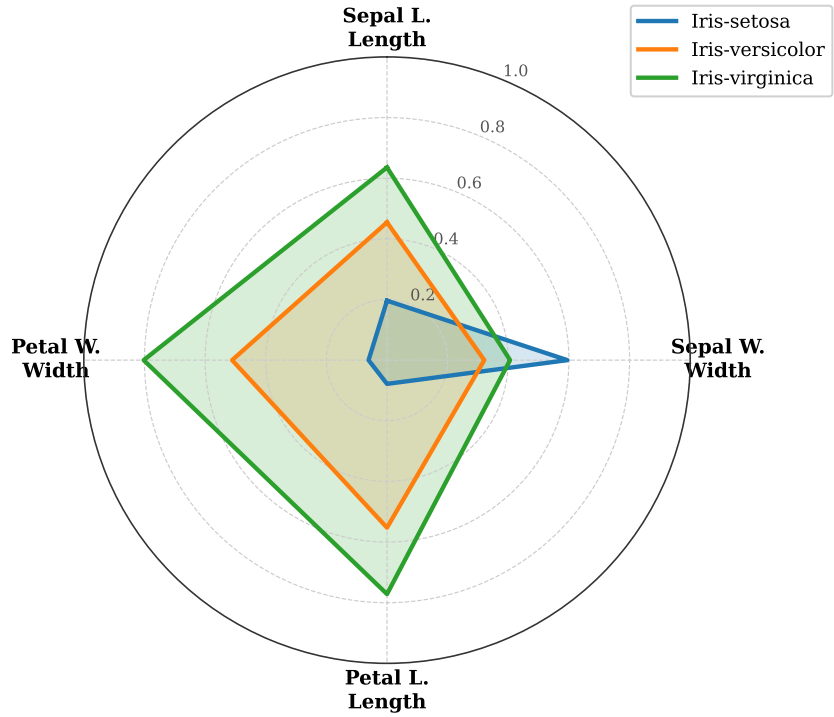


Figure 3.4: Class-average profiles for the Iris dataset rendered via Radar Plot.

3.2 Bivariate Dependence Analysis: Covariance and Correlation

Quantitative dependence analysis investigates the strength and direction with which two random variables co-vary.

3.2.1 From Variance to Covariance

While variance measures the dispersion of a single variable around its own expectation, **Sample Covariance** measures the joint tendency of two numerical variables X and Y to deviate synchronously from their sample means:

$$\text{Cov}(X, Y) = s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) \quad (3.2)$$

In the special case where $X = Y$, covariance equals sample variance:

$$\text{Cov}(X, X) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 = s_x^2 \quad (3.3)$$

Covariance Matrix

For a set of p continuous attributes, joint variability is completely captured by the **Covariance Matrix** $V \in \mathbb{R}^{p \times p}$:

$$V = \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_p) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) & \cdots & \text{Cov}(X_2, X_p) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_p, X_1) & \text{Cov}(X_p, X_2) & \cdots & \text{Cov}(X_p, X_p) \end{bmatrix} \quad (3.4)$$

- Diagonal entries $V_{jj} = \text{Var}(X_j) = s_j^2$ represent marginal sample variances.
- Off-diagonal entries $V_{jk} = \text{Cov}(X_j, X_k)$ represent pairwise covariances.
- Since $\text{Cov}(X_j, X_k) = \text{Cov}(X_k, X_j)$, matrix V is **symmetric** ($V = V^T$) and positive semi-definite ($V \succeq 0$).

Geometric Interpretation of Deviation Products

The sign of sample covariance is governed by the quadrants formed around the bivariate mean $(\bar{x}, \bar{y})$ in Cartesian space:

- Points in Quadrants I and III contribute positive products (concordant deviations) $\implies \text{Cov}(X, Y) > 0$.

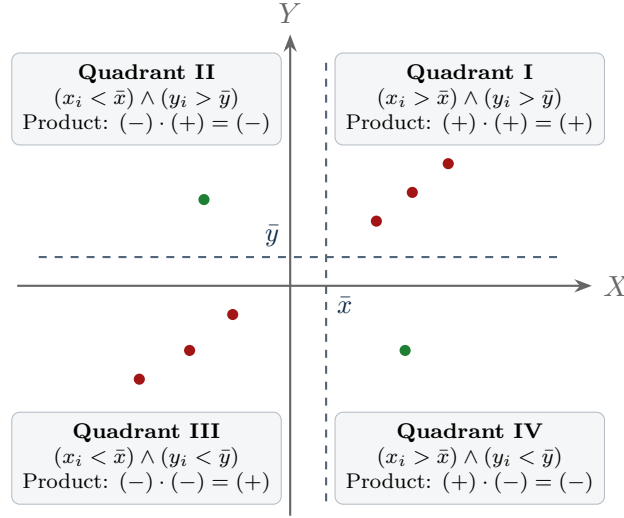


Figure 3.5: Geometric decomposition of bivariate deviation products across four mean-centered quadrants.

- Points in Quadrants II and IV contribute negative products (discordant deviations) $\implies \text{Cov}(X, Y) < 0$.
- Uniformly dispersed observations cancel out across quadrants, yielding $\text{Cov}(X, Y) \approx 0$.

Structural Limitation: Scale Dependence

Covariance lacks scale invariance: its magnitude scales linearly with measurement units. Converting income X from euros to thousands of euros ($X' = X/1000$) shrinks covariance by a factor of 1000, although the underlying statistical link remains identical. Hence, covariance cannot directly compare pairs with distinct units or scales.

3.2.2 Pearson Linear Correlation Coefficient

The **Pearson Linear Correlation Coefficient** (r_{xy}) standardizes covariance by dividing by the product of both sample standard deviations:

$$r_{xy} = \frac{\text{Cov}(X, Y)}{s_x s_y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (3.5)$$

Mathematical Properties of r_{xy}

1. **Bounded Range:** By the Cauchy-Schwarz inequality on centered vectors, the coefficient is strictly bounded within:

$$-1 \leq r_{xy} \leq +1 \quad (3.6)$$

2. **Scale Invariance:** It is a dimensionless index invariant under positive affine transformations ($X' = aX + b, Y' = cY + d$ with $a, c > 0$).

3. **Perfect Linear Relationship:**

- $r_{xy} = +1$: perfect positive linear correlation; all points lie exactly on a line with positive slope ($y = ax + b, a > 0$).
- $r_{xy} = -1$: perfect negative linear correlation ($a < 0$).

4. **Absence of Linear Relationship:** A value of $r_{xy} \approx 0$ signifies **absence of linear association**, but does not imply statistical independence. Non-linear deterministic functions (e.g., $Y = X^2$ with X symmetrically distributed around zero) yield $r_{xy} = 0$, despite full underlying determinism.

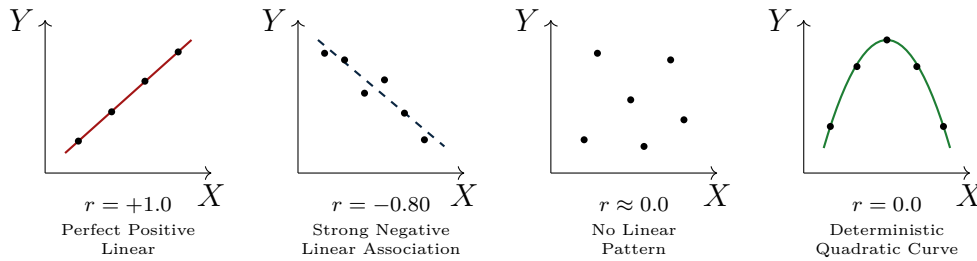


Figure 3.6: Archetypal scatter patterns and their corresponding Pearson correlation values.

3.2.3 Causal Fallacy and Confounding Variables

A cornerstone of data science states that **correlation does not imply causation**. A high observed $|r_{xy}|$ may arise from three distinct phenomena:

1. X is a direct causal driver of Y .
2. Y is a direct causal driver of X .

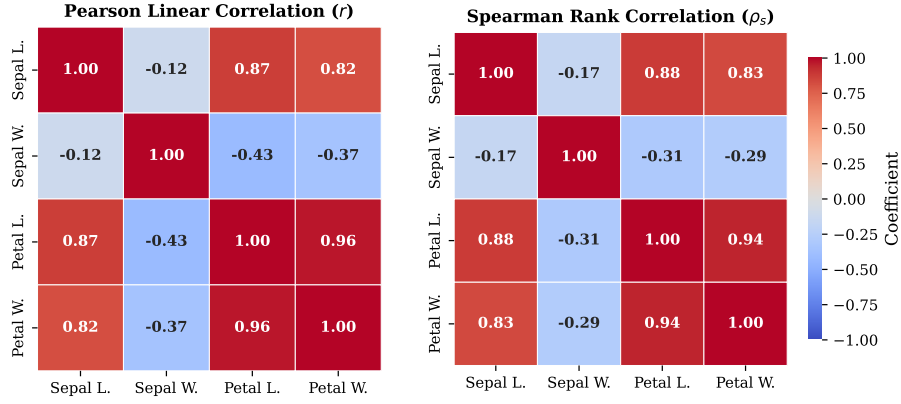


Figure 3.7: Correlation heatmaps for the overall Iris dataset and stratified by species.

3. X and Y are simultaneously driven by an unobserved or latent third factor, termed a **confounding variable**.

Classic Example: Ice Cream Sales and Drowning Incidents

During summer months, a high positive correlation ($r > 0.85$) is documented between ice cream sales (X) and drowning casualties (Y). Inferring that ice cream consumption increases drowning risk represents a classic causal fallacy: both variables are driven by an underlying exogenous confounder—high atmospheric temperature (*hot weather*)—which spurs both outdoor swimming and cold dessert purchases.

3.2.4 Spearman Rank Correlation Coefficient

When variables violate bivariate normality assumptions or exhibit non-linear yet strictly monotonic relationships, the **Spearman Rank Correlation Coefficient** (ρ) is preferred.

The method replaces continuous values x_i and y_i with their corresponding ordinal ranks $r(x_i), r(y_i) \in \{1, \dots, n\}$. In the absence of ties, the explicit formulation simplifies to:

$$\rho = 1 - \frac{6 \sum_{i=1}^n (r(x_i) - r(y_i))^2}{n(n^2 - 1)} \quad (3.7)$$

- $\rho = +1$: indicates a perfect monotonic increasing relationship (as X increases, Y strictly increases, even along exponential or logarithmic curves).

- $\rho = -1$: indicates a perfect monotonic decreasing relationship.
- **Robustness**: Relying on ranks rather than raw metric differences renders Spearman significantly more resistant to extreme outliers in distribution tails.

3.3 Outlier Taxonomy and Detection Methodologies

An **outlier** is defined as an observation that deviates so substantially from the remaining data as to arouse suspicion that it was generated by a different mechanism or contaminated by measurement error.

3.3.1 The Dual Nature of Outliers: Noise vs. Primary Target

- **Outliers as Noise**: The anomaly stems from data entry typos, temporary sensor glitches, or spurious noise. In this role, outliers distort parameter estimation and should be identified, corrected, or removed during data preprocessing.
- **Outliers as the Primary Target (Analysis Goal)**: The explicit mission of the analysis is to uncover rare, high-impact events carrying crucial domain insights:
 - Intrusion detection in computer network security.
 - Financial fraud detection and anti-money laundering monitoring.
 - Predictive maintenance through anomalous vibration patterns in industrial turbines.

3.3.2 Outlier Identification Methodologies

Outlier identification methodologies can be systematically categorized into univariate and multidimensional approaches:

- **Categorical Attributes**: An anomalous observation belongs to a rare category occurring with near-zero empirical frequency ($< 0.1\%$).
- **Univariate Numeric Attributes**:
 - *Tukey IQR Rule*: Values outside the interval $[Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]$.

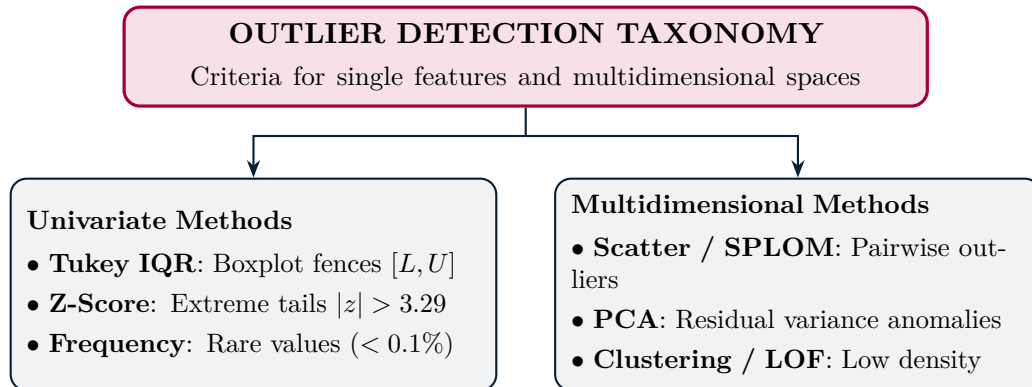


Figure 3.8: Taxonomy of outlier detection methodologies.

- *Z-Score Rule*: Standardized values with $|z| > 3.29$ (representing the outermost 0.1% tails of a Gaussian distribution).
- **Multidimensional Spaces**:
 - *Bivariate Scatter Plots*: Visual identification of points detached from dense clusters.
 - *PCA Projections*: Observations projecting as outliers along secondary principal components.
 - *Clustering and Local Outlier Factor (LOF)*: Isolating samples residing in low-density neighborhoods relative to their k -nearest neighbors.

3.4 Introduction to Missing Values

A data instance contains **Missing Values** when no observation is recorded for one or more features (explicitly encoded via symbols like NaN, NULL, ? or implicitly hidden under sentinel conventions such as 999 or 0).

3.4.1 Root Causes of Missingness

1. **Voluntary Non-Response**: Respondents decline to disclose sensitive personal data (e.g., annual income, political affiliations, medical history).
2. **Conceptual Inapplicability**: The attribute is logically inapplicable for certain subpopulations (e.g., pregnancy counts for male subjects, or employment tenure for school-age minors).

3. **Hardware Failures or Dropped Connections:** Telemetry sensors freezing or disconnecting, producing null readings or reporting continuous dummy constants (e.g., a frozen wind sensor persistently reporting 0 m/s).

3.4.2 Operational Data Understanding Checklist

Before proceeding to data cleaning and preparation, practitioners must systematically verify the following diagnostic checklist:

Diagnostic Check	Operational Verification Procedure
Syntactic Integrity	Validate data types, string/number formats, and domain constraints.
Outlier Inspection	Inspect extreme values using univariate Boxplots and bivariate Scatter Plots.
Missing Value Audit	Quantify null percentages and detect implicit sentinel values (0, -1, 999).
Association & Redundancy	Compute Pearson and Spearman matrices to identify collinear or redundant features.
Distributional Assumptions	Check symmetry, normality, and variance stability prior to feature transformations.

Table 3.1: Operational diagnostic checklist for completing the Data Understanding phase.

Chapter 4

Data Preparation & Cleaning – Reduction, Discretization, and Transformation

While Data Understanding focuses on diagnosing anomalies and comprehending latent structures within a dataset, **Data Preparation** represents the operational, interventional component of the KDD lifecycle. During this phase, raw records are cleansed, compressed, scaled, and re-encoded to satisfy the algebraic and computational requirements of machine learning and data mining algorithms.

4.1 Distinction Between Data Understanding and Data Preparation

The transition from diagnostic exploration to operational data engineering is captured by the clear mapping between diagnostic findings and corresponding corrective actions:

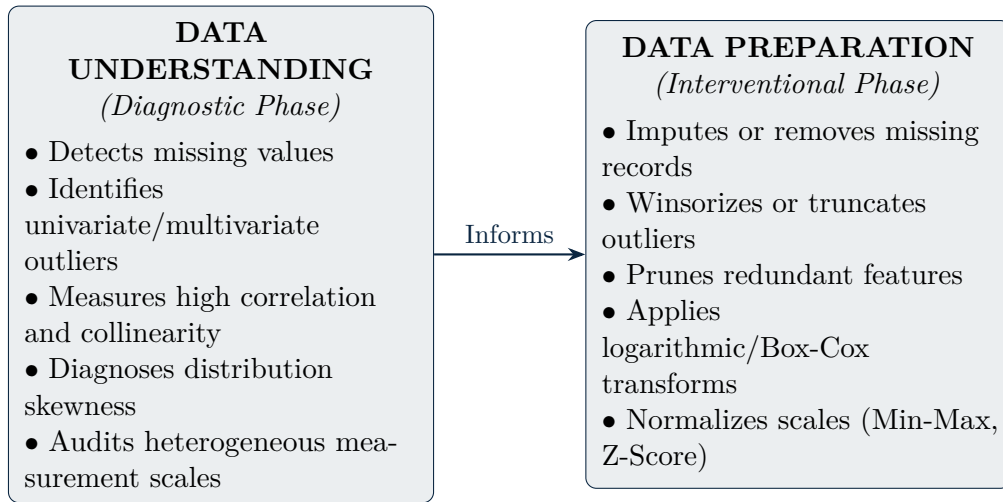


Figure 4.1: Sequential transition from Data Understanding (diagnosis) to Data Preparation (intervention).

4.2 Aggregation and Sampling

Reducing dataset volume can be achieved either by combining related observations or through statistical sampling of representative subsets.

4.2.1 Data Aggregation

Data aggregation merges two or more records or features into a single synthesized summary.

- **Core Objectives:**
 1. *Dimensional Reduction*: Reduces the total record count or feature cardinality, lowering computational complexity.
 2. *Scale Transformation*: Adjusts analysis to an appropriate conceptual level of granularity (e.g., aggregating hourly transactional logs into weekly sums, or regionalizing individual point-of-sale terminals).
 3. *Variance Stabilization*: Aggregated aggregates average out high-frequency noise, yielding lower-variance metrics and more robust statistical trends.
- **Case Study: Australian Precipitation (1982–1993)**: In historical Australian rainfall data, the standard deviation of monthly average

rainfall exhibits high dispersion with long tails reaching up to 15 cm. Conversely, when aggregating into annual averages, the standard deviation tightly clusters around low values with substantially reduced spread (Figure 4.2). To model seasonal agricultural patterns, the disaggregated monthly scale is essential, whereas comparing macroscopic regional climates benefits from the stable annual scale.

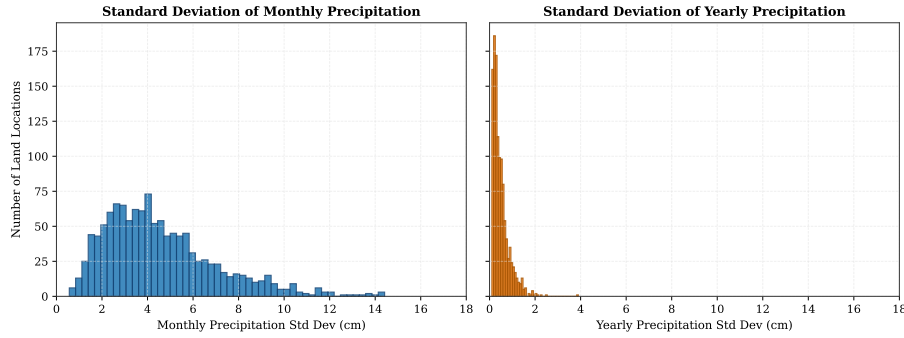


Figure 4.2: Dispersion comparison between monthly precipitation and aggregated annual precipitation in Australia.

4.2.2 Sampling Methodologies

Sampling selects a representative subset of size n from a large population of size N ($n \ll N$). A sample is deemed **representative** if it preserves the underlying statistical properties and distributional profile of the parent population.

1. **Simple Random Sampling:** Every instance in the parent dataset has identical a priori inclusion probability:

$$P(x_i) = \frac{1}{N} \quad (4.1)$$

- *Without Replacement:* Extracted items are removed from the candidate pool and cannot be drawn again.
 - *With Replacement:* Extracted items remain in the pool and may appear multiple times in the sample.
2. **Stratified Sampling:** The dataset is partitioned into mutually disjoint, homogeneous subsets called *strata* (e.g., grouped by target class label or numeric percentiles). Independent random samples are extracted from each stratum proportional to its relative population size, preventing rare classes from being erased.

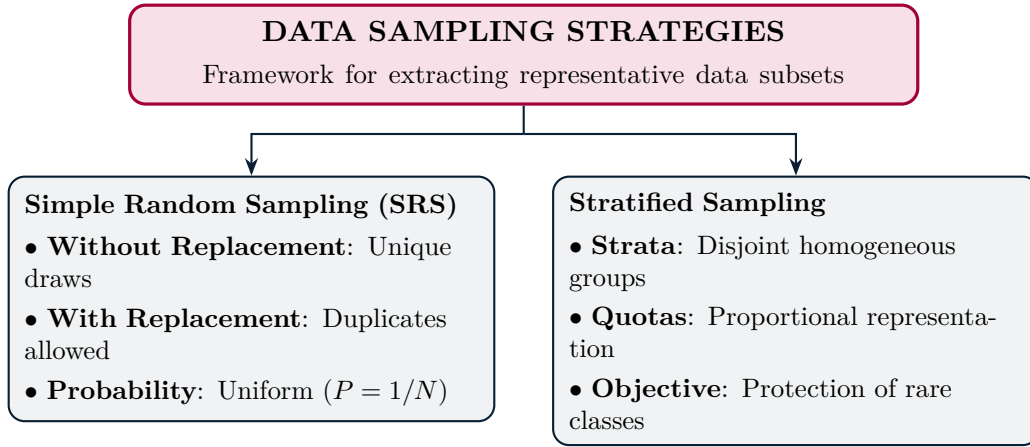


Figure 4.3: Taxonomy of data sampling strategies in data mining.

Selecting an optimal sample size involves a trade-off between computational processing cost and statistical fidelity, as depicted in Figure 4.4.

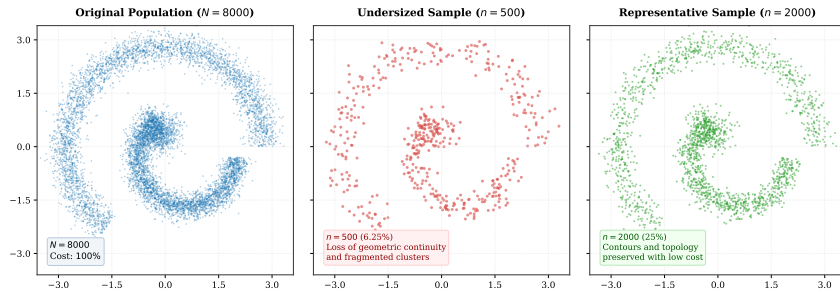


Figure 4.4: Trade-off curve between sample size and empirical parameter estimation error.

4.3 Dimensionality Reduction

High feature counts introduce significant computational and geometric bottlenecks.

4.3.1 The Curse of Dimensionality

As feature dimensionality p grows, the geometric volume of the embedding space $\mathbb{R}^p$ expands exponentially relative to the available sample count. Key consequences include:

1. **Extreme Sparsity:** Data observations become isolated in high-dimensional space; local point density vanishes towards zero, leaving the feature space virtually empty.
2. **Distance Concentration:** As $p \rightarrow \infty$, the contrast between the maximum Euclidean distance and minimum Euclidean distance to any query point collapses:

$$\lim_{p \rightarrow \infty} \frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} \rightarrow 0 \quad (4.2)$$

This severely impairs distance-based algorithms (k -NN, K-Means clustering, local density estimators).

3. **Covariance Instability:** Reliably estimating a $p \times p$ sample covariance matrix requires sample sizes that scale exponentially with p to avoid rank deficiency and ill-conditioning.

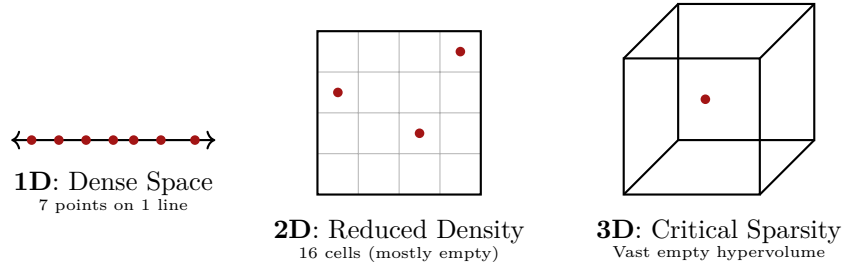


Figure 4.5: Visualizing exponential geometric dispersion of data samples with increasing dimensions.

4.3.2 Feature Selection

Feature Selection isolates an optimal subset of original attributes by discarding irrelevant (uninformative) and redundant (duplicated) features:

4.3.3 Feature Creation and Projection

- **Feature Construction:** Formulates novel descriptive attributes by combining raw features using domain knowledge (e.g., calculating density as mass/volume or productivity as completed/scheduled hours).

Paradigm	Selection Mechanism	Trade-offs
Filter	Ranks features a priori using univariate statistical metrics (ANOVA, Pearson r , χ^2), independent of learning models.	Blazing fast; ignores multivariate feature interactions.
Wrapper	Uses the target machine learning model as an evaluation oracle across candidate subsets (Forward Selection, Backward Elimination).	High predictive accuracy; computationally heavy (2^p potential subset evaluations).
Embedded	Selection occurs natively during algorithm optimization (e.g., decision tree branch splits, Lasso L_1 penalty regularization).	Excellent balance of computational efficiency and predictive accuracy.

Table 4.1: Comparative breakdown of Filter, Wrapper, and Embedded feature selection approaches.

- **Feature Projection:** Algebraically projects original vectors from $\mathbb{R}^p$ into a compact subspace $\mathbb{R}^k$ ($k \ll p$), preserving maximal variance or class separation (PCA, SVD, Linear Discriminant Analysis - LDA, Autoencoders).

4.4 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a linear, unsupervised technique that maps an original set of correlated features into a new coordinate system of mutually orthogonal (uncorrelated) variables, ordered in descending fashion by explained variance.

4.4.1 Step-by-Step PCA Mathematical Derivation

The formal mathematical workflow of PCA proceeds across five structured steps:

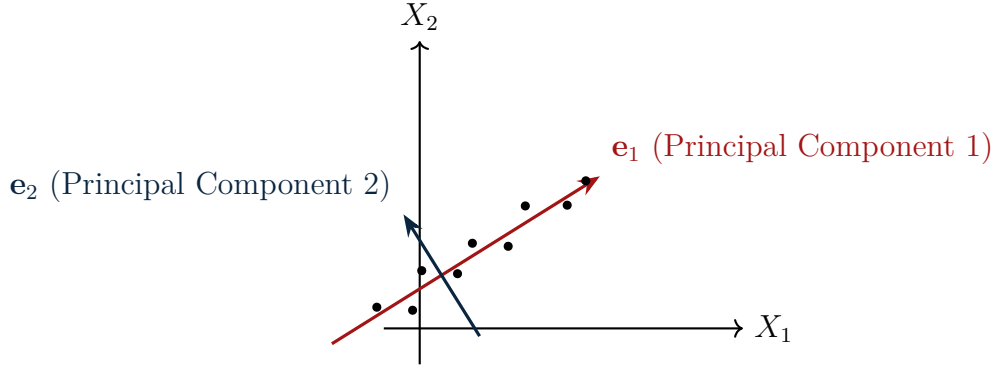


Figure 4.6: Alignment of orthogonal principal component axes along directions of maximal data variance.

Step 1: Centering and Standardizing the Data Matrix

Given a data matrix $D \in \mathbb{R}^{n \times p}$, compute the sample mean for each feature column:

$$\bar{x}_j = \frac{1}{n} \sum_{i=1}^n D_{ij} \quad (4.3)$$

Construct the **centered matrix** $C \in \mathbb{R}^{n \times p}$ by subtracting feature means from each observation:

$$C_{ij} = D_{ij} - \bar{x}_j \quad (4.4)$$

If attributes exhibit dissimilar scales or measurement units, construct the **standardized matrix** $Z \in \mathbb{R}^{n \times p}$ by dividing centered residuals by feature standard deviations s_j :

$$Z_{ij} = \frac{D_{ij} - \bar{x}_j}{s_j} \quad (4.5)$$

Step 2: Sample Covariance Matrix Computation

Compute the empirical covariance matrix $V \in \mathbb{R}^{p \times p}$ via matrix multiplication:

$$V = \frac{1}{n-1} C^T C \quad (4.6)$$

Step 3: Spectral Decomposition (Eigenvalues and Eigenvectors)

Solve the characteristic eigenvalue equation for symmetric matrix V :

$$V \mathbf{e}_j = \lambda_j \mathbf{e}_j, \quad j = 1, \dots, p \quad (4.7)$$

- Each normalized unit-length **eigenvector** $\|\mathbf{e}_j\|_2 = 1$ establishes the spatial direction of the j -th principal axis. Because V is symmetric, eigenvectors belonging to distinct eigenvalues are mutually orthogonal ($\mathbf{e}_j^T \mathbf{e}_k = 0$ for $j \neq k$).
- Each corresponding **eigenvalue** $\lambda_j \geq 0$ quantifies the exact portion of sample variance explained along the axis $\mathbf{e}_j$.

Step 4: Sorting and Component Retention

Sort eigenvalues and associated eigenvectors in descending numerical order:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0 \quad (4.8)$$

The proportion of total variance explained by the j -th component equals:

$$\text{Explained Variance (\%)} = \frac{\lambda_j}{\sum_{m=1}^p \lambda_m} \times 100 \quad (4.9)$$

Retain the top k components ($k < p$) satisfying an aggregate cumulative variance threshold (e.g., 85% or 95%), discarding residual trailing components (Figure 4.7). Construct the projection matrix $F \in \mathbb{R}^{p \times k}$:

$$F = [\mathbf{e}_1 \quad \mathbf{e}_2 \quad \dots \quad \mathbf{e}_k] \quad (4.10)$$

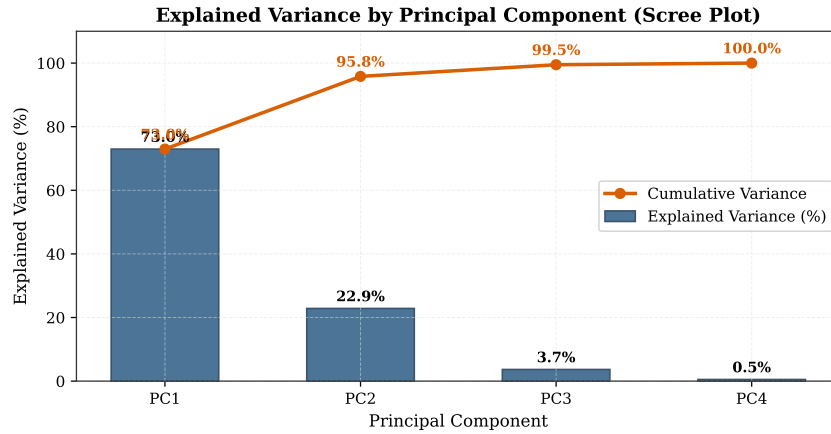


Figure 4.7: Eigenvalue scree plot and cumulative explained variance curve.

Step 5: Subspace Projection

Project the centered dataset onto the reduced k -dimensional coordinate system:

$$Z = CF \quad (4.11)$$

Figure 4.8 illustrates the 2D projection of the Iris dataset onto its first two principal components (PC1 and PC2), which capture over 95% of total dataset variance while achieving distinct species clustering.

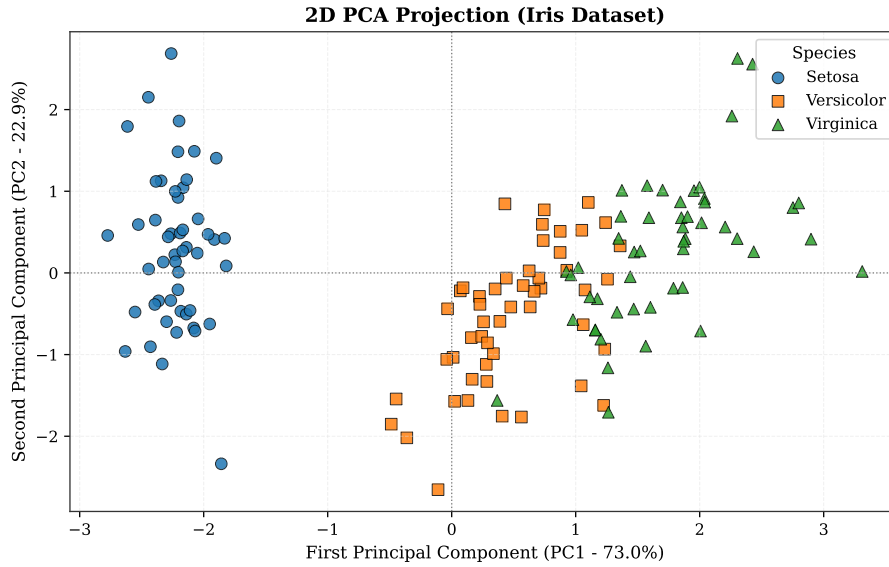


Figure 4.8: Two-dimensional projection of Iris samples onto the top two principal components.

4.5 Handling Missing Values

Operational strategies for incomplete records include:

1. **Listwise Deletion:** Discarding all records containing at least one missing attribute. Safe only when missing frequency is trivial ($< 3-5\%$) and mechanism is strictly *Missing Completely at Random (MCAR)*; otherwise, severe sampling bias ensues.
2. **Central Tendency Imputation:**
 - *Mean:* For symmetric numeric variables lacking outliers.

- *Median*: For skewed numeric features with heavy tails or outliers.
 - *Mode*: For categorical attributes.
 - *Caveat*: Artificially deflates feature variance and weakens inter-attribute correlation structures.
3. **Conditional Segmented Imputation**: Grouping instances into clusters or classes based on complete features, then imputing missing values using the mean or median calculated within that homogeneous segment.
 4. **Model-Based Predictive Imputation**: Fitting a supervised model (k -NN, linear regression, Random Forest) where the missing feature acts as the target variable and complete attributes serve as predictors.

4.6 Discretization and Binarization Techniques

Discretization converts continuous quantitative variables into discrete ordinal attributes by partitioning the continuous support into a finite set of contiguous intervals (termed *bins*).

4.6.1 Unsupervised Discretization

1. Natural Binning (Equal-Width Binning)

Partitions the range $[x_{\min}, x_{\max}]$ into k equal-sized intervals of **constant width**:

$$\delta = \frac{x_{\max} - x_{\min}}{k} \quad (4.12)$$

The i -th bin ($i = 0, \dots, k - 1$) spans:

$$I_i = [x_{\min} + i\delta, x_{\min} + (i + 1)\delta) \quad (4.13)$$

Numerical Example: 12 Beer Prices

Given $N = 12$ beer prices spanning from $x_{\min} = 100$ to $x_{\max} = 160$, choosing $k = 4$ bins yields interval width $\delta = \frac{160-100}{4} = 15$. Intervals and counts:

- Bin 1: $[100, 115) \rightarrow \{100, 110, 110\}$ (Count: 3)
- Bin 2: $[115, 130) \rightarrow \{120, 120, 125\}$ (Count: 3)
- Bin 3: $[130, 145) \rightarrow \{130, 130, 135, 140\}$ (Count: 4)
- Bin 4: $[145, 160] \rightarrow \{150, 160\}$ (Count: 2)

2. Equal-Frequency Binning (Equal-Depth Binning)

Sorts the data ascendingly and allocates an equal number of records into each bin:

$$f = \frac{N}{k} \quad (4.14)$$

For the same sorted 12 prices $\{100, 110, 110, 120, 120, 125, 130, 130, 135, 140, 150, 160\}$ with $k = 4$, target count is $f = 12/4 = 3$:

- Bin 1: $\{100, 110, 110\}$
- Bin 2: $\{120, 120, 125\}$
- Bin 3: $\{130, 130, 135\}$
- Bin 4: $\{140, 150, 160\}$

This balances density and dampens outlier skewness, though identical values straddling bin boundaries may be artificially split into adjacent categories.

Formulas for Optimal Bin Count and Width

Beyond Sturges' formula, the base-10 logarithmic rule provides:

$$C = 1 + \frac{10}{3} \log_{10}(N) \quad (4.15)$$

To optimize bin width h accounting jointly for sample spread and size, **Scott's Rule (1979)** provides:

$$h = \frac{3.5 \times s}{\sqrt[3]{N}} \quad (4.16)$$

where s is the sample standard deviation and N is sample size.

4.6.2 Supervised Entropy-Based Discretization

When a target class label $Y \in \{c_1, \dots, c_m\}$ is available, split thresholds are selected to maximize class homogeneity. The Shannon entropy of dataset S is:

$$\text{Entropy}(S) = - \sum_{j=1}^m p_j \log_2(p_j) \quad (4.17)$$

where p_j is the proportion of samples belonging to class c_j . A candidate threshold T partitioning S into S_1 and S_2 yields weighted entropy:

$$E(S, T) = \frac{|S_1|}{|S|} \text{Entropy}(S_1) + \frac{|S_2|}{|S|} \text{Entropy}(S_2) \quad (4.18)$$

The optimal split T^* maximizes information gain $\text{Gain}(S, T) = \text{Entropy}(S) - E(S, T)$.

4.6.3 Binarization (Encoding)

Converts categorical attributes with m distinct levels into binary indicators $\{0, 1\}$:

- **One-Hot Encoding (Asymmetric Binary Encoding)**: Allocates a dedicated indicator feature for each category level. Avoids imposing artificial order:

Category	Integer Code	x_1	x_2	x_3	x_4	x_5
awful	0	1	0	0	0	0
poor	1	0	1	0	0	0
OK	2	0	0	1	0	0
good	3	0	0	0	1	0
great	4	0	0	0	0	1

Table 4.2: One-Hot encoding for an attribute with 5 distinct qualitative categories.

- **Compact Binary Encoding**: Encodes category levels into positional binary integers using $n = \lceil \log_2(m) \rceil$ bits: *Drawback*: Induces synthetic

Category	Integer Code	x_1	x_2	x_3
awful	0	0	0	0
poor	1	0	0	1
OK	2	0	1	0
good	3	0	1	1
great	4	1	0	0

Table 4.3: Compact binary encoding on 3 bits for 5 categories.

correlations and ordering artifacts among bits that lack real-world domain validity.

4.7 Functional Attribute Transformations and Variance Stabilization

A **functional transformation** applies a strictly monotonic, continuous function T to every value of feature X : $Y = T(X)$.

- **Variance Stabilization:** Enforces constant error variance across attribute ranges (*homoscedasticity*).
- **Distribution Normalization:** Transforms heavily skewed distributions toward approximate Gaussianity.
- **Relationship Linearization:** Transforms curvilinear relationships into linear ones suitable for correlation and regression.

4.7.1 Power and Logarithmic Transformation Family

The general parametric transformation is formulated as:

$$T_p(x) = \begin{cases} ax^p + b & \text{if } p \neq 0 \\ c \log(x) + d & \text{if } p = 0 \end{cases} \quad (4.19)$$

with $x > 0$.

- **Linear Transformation** ($p = 1$): Alters only scale and origin without changing distribution shape (e.g., Fahrenheit to Celsius: $a = 5/9, b = -160/9$).
- **Logarithmic Transformation** ($p = 0$): $T(x) = c \log(x) + d$. Compresses extreme upper-tail values while stretching small values, effectively correcting severe right skewness.

Bar	Beer	Original Gain (X)	Transformed Gain $\log_{10}(X)$
A	Bud	20	1.3010
A	Becks	10000	4.0000
C	Bud	300	2.4771
D	Bud	400	2.6021
D	Becks	5	0.6990
E	Becks	120	2.0792
E	Bud	120	2.0792
F	Bud	11000	4.0414
G	Bud	1300	3.1139
H	Bud	3200	3.5051
H	Becks	1000	3.0000
I	Bud	135	2.1303

Table 4.4: Empirical sample of 12 commercial Gain records before and after $\log_{10}$ transformation.

Statistical Metric	Raw Scale (X)	Logarithmic Scale ($Y = \log_{10} X$)
Mean	2481.82	2.5956
Standard Deviation	4079.02	1.0651
Minimum	5	0.6990
1 st Quartile (Q_1)	120	2.0792
Median (Q_2)	400	2.6021
3 rd Quartile (Q_3)	2250	3.3095
Maximum	11000	4.0414

Table 4.5: Descriptive metrics before and after logarithmic transformation.

In the raw scale, standard deviation (4079.02) dwarfs the mean (2481.82), heavily pulled by outliers at 10000 and 11000. Following $\log_{10}$ scaling, mean (2.5956) and median (2.6021) align closely, restoring symmetry.

4.7.2 Other Variance Stabilizing Transformations

- **Square Root** ($p = 1/2$): $T(x) = \sqrt{x}$. Stabilizes variance for Poisson count data ($\sigma^2 = \mu$).
- **Reciprocal** ($p = -1$): $T(x) = 1/x$. Applied when variance increases superlinearly relative to the mean.

4.8 Normalization Algorithms

Normalization scales diverse numerical features onto a shared, comparable range, preventing features with large magnitudes from dominating distance computations in k -NN, K-Means, SVM, or artificial neural networks.

4.8.1 Min-Max Normalization

Linearly transforms values from original range $[\min_A, \max_A]$ into a defined target interval $[\text{new_min}_A, \text{new_max}_A]$ (typically $[0, 1]$):

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A \quad (4.20)$$

Preserves exact linear proportions, but remains vulnerable to out-of-bounds future inputs and extreme outliers compressing standard values.

4.8.2 Z-Score Standardization

Shifts and scales values such that the transformed variable has **zero mean and unit variance**:

$$v' = \frac{v - \bar{x}_A}{s_A} \quad (4.21)$$

where $\bar{x}_A$ is the sample mean and s_A the sample standard deviation. It is the gold standard when data approximates a Gaussian distribution.

4.8.3 Decimal Scaling

Normalizes values by shifting the decimal point via powers of 10:

$$v' = \frac{v}{10^j} \quad (4.22)$$

where j is the smallest positive integer satisfying:

$$\max(|v'|) < 1 \iff 10^{j-1} \leq \max(|v|) < 10^j \quad (4.23)$$

Numerical Example: Decimal Scaling

Given the vector $V = \{-10, 201, 301, -401, 501, 601, 701\}$:

1. The absolute maximum is $m = \max(|v|) = 701$.
2. The minimal integer j where $10^j > 701$ is $j = 3$ ($10^3 = 1000$).
3. Dividing each value by 1000 yields:

$$V' = \{-0.01, 0.201, 0.301, -0.401, 0.501, 0.601, 0.701\} \quad (4.24)$$

Every transformed element strictly lies within the open interval $(-1, +1)$.

4.9 Univariate Outlier Treatment

Operational treatment of isolated univariate outliers relies on formal detection and replacement criteria:

4.9.1 Interquartile Criterion (Tukey)

1. Compute interquartile range $IQR = Q_3 - Q_1$.

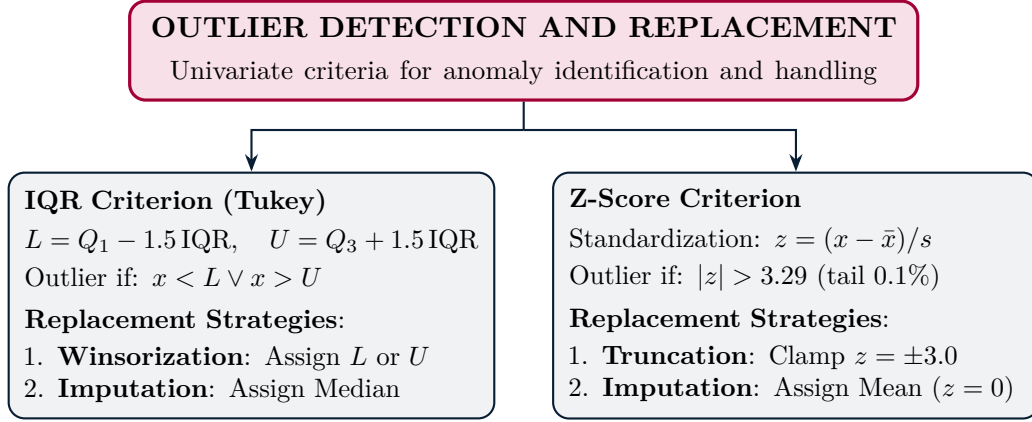


Figure 4.9: Decision hierarchy for univariate outlier detection and replacement strategies.

2. Define inner fences:

$$L = Q_1 - 1.5 \times \text{IQR}, \quad U = Q_3 + 1.5 \times \text{IQR} \quad (4.25)$$

3. Observation x is flagged as an outlier if $x < L \vee x > U$.

4. **Treatment:**

- *Winsorization:* Clamp x to L if $x < L$, or to U if $x > U$. Preserves ordinal rankings while neutralizing distortive tail extremes.
- *Median Imputation:* Replace flagged outliers with the sample median computed from non-outlier records.

4.9.2 Z-Score Criterion

Under standard normal assumptions, 95% of samples fall within $|z| \leq 1.96$, 99% within $|z| \leq 2.58$, and 99.9% within $|z| \leq 3.29$. Flag as outliers instances exceeding:

$$|z| > 3.29 \iff x < \bar{x} - 3.29 s \quad \vee \quad x > \bar{x} + 3.29 s \quad (4.26)$$

Replacement Strategies

1. **3 σ Truncation:** Clamp extreme values to boundary $z = \pm 3.0$ ($x' = \bar{x} \pm 3.0 s$), pulling observations into plausible bounds without erasing their status as extreme observations.

2. **Mean Imputation:** Substitute the outlier with $z = 0$, equivalent to the sample mean $\bar{x}$.

Chapter 5

Python Lab: Data Understanding and Data Preparation

This chapter presents the computational realization of the theoretical principles of Data Understanding (DU) and Data Preparation (DP), drawing upon the practical laboratory exercises implemented with Python's core data science ecosystem (`numpy`, `pandas`, `scipy`, `matplotlib`, `seaborn`, and `scikit-learn`).

The laboratory uses the **Tips** dataset (augmented with the secondary **Tips_Age** table), describing restaurant customer transactions and tipping behaviors, to demonstrate the complete data engineering workflow: from ingestion, relational joining, and deduplication to missing value diagnosis, conditional imputation, feature engineering, normalization, and dimensionality reduction via PCA and *t*-SNE.

Interactive Environment and Laboratory Reproducibility

All examples, data preparation pipelines, and visualizations presented in this chapter are fully reproducible and interactively executable via Jupyter notebooks located in the `Data-Mining/notebooks/` directory.

Quick start with Docker:

- Terminal execution (project root): `docker compose up jupyter`
- Windows shortcut: double-click `scripts/start-jupyter.cmd`
- Web interface: <http://localhost:8888/lab> (no token or password required).

To execute headless Python scripts directly from the command line:

- `docker compose run -rm app python3 Data-Mining/notebooks/generate_dispensa_figures.py`

5.1 Environment, Ingestion, and Data Integration

Remark 5.1.1 (Interactive Notebook Reference). Code examples in Sections 5.1, 5.2, and 5.3 correspond to the interactive Jupyter **Notebook 1 (Data Understanding)** (file `1_basics_and_understanding.ipynb`).

The scientific software stack employed throughout the laboratory exercises comprises:

```
1 import numpy as np
2 import pandas as pd
3 import scipy.stats as stats
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.impute import SimpleImputer, KNNImputer
7 from sklearn.preprocessing import StandardScaler,
   MinMaxScaler, KBinsDiscretizer, OneHotEncoder,
   TargetEncoder
8 from sklearn.decomposition import PCA
9 from sklearn.manifold import TSNE
```

5.1.1 Loading and Structural Audit

The primary dataset is ingested from a tab-separated values (tsv) file, setting the index to the first column:

```
1 tips = pd.read_csv('tips.csv', sep='\t', index_col=0)
2 print(tips.head())
3 tips.info()
```

The raw data initially contains 244 observations and 7 features: **total_bill** (total bill amount), **tip** (tip amount), **sex** (server/payer gender), **smoker** (smoking party indicator), **day** (day of week), **time** (meal timing: Lunch or Dinner), **size** (party size).

5.1.2 Data Integration and Relational Joining

In this lab, customer age data is stored in a separate table (**tips_age.csv**). The two sources are integrated using a left-join keyed on the shared primary identifier ID:

```
1 tips_age = pd.read_csv('tips_age.csv', sep='\t',
2                       index_col=0)
3 df = tips.join(tips_age, on='ID')
4 # Drop redundant columns originating from the secondary
   source
5 df = df.drop(columns=['gender'], errors='ignore')
```

5.1.3 Duplicate Row Detection and Filtering

Identifying duplicate rows requires inspecting exact equality across all attribute dimensions:

```
1 # Display all duplicate rows (keeping all duplicate
   instances)
2 duplicated_rows = df[df.duplicated(keep=False)]
3 print(f"Duplicate rows detected: {len(duplicated_rows)}")
4
5 # Drop duplicate rows while retaining the first
   occurrence
6 df = df.drop_duplicates()
```

5.1.4 Cleaning Invalid Values and Type Casting

Real-world datasets frequently contain formatting anomalies or sentinel strings mistakenly entered into numeric fields (such as the dummy string 'aaaaa' in the `total_bill` column). This forces Pandas to treat the column as a generic object type. Cleaning requires string replacement followed by explicit numerical casting:

```
1 # Replace invalid string with NaN
2 df["total_bill"] = df["total_bill"].replace("aaaaa", np.
      nan)
3
4 # Cast column to numeric float
5 df["total_bill"] = pd.to_numeric(df["total_bill"])
```

5.2 Missing Value Diagnosis and Imputation

5.2.1 Missing Value Audit

Missing values (NaN) are systematically audited by chaining `isnull()` and `sum()`:

```
1 # Summary of null counts per column
2 null_counts = df.isnull().sum()
3 print(null_counts[null_counts > 0])
4
5 # Filter records containing at least one missing
      attribute
6 incomplete_records = df[df.isnull().any(axis=1)]
```

5.2.2 Deletion vs. Statistical Imputation

- **Listwise Deletion:** Discarding all rows containing one or more null values:

```
1 df_cleaned = df.dropna()
```

- **Conditional Median Imputation:** When the `age` feature exhibits missing values or dummy zeros (0), global median imputation can mask real demographic disparities between meal times (*Lunch* vs. *Dinner*). Group-stratified imputation preserves local distributions:

```

1 df['age'] = df['age'].replace(0, np.nan)
2 # Impute using group-conditional median based on
  meal time
3 df['age'] = df['age'].fillna(df.groupby('time')['age']
  ).transform('median'))

```

5.2.3 Advanced Imputation with Scikit-Learn

The `sklearn.impute` module provides transformer objects suitable for production pipelines:

```

1 # 1. SimpleImputer for numerical attributes (mean or
  median)
2 imp_mean = SimpleImputer(strategy='mean')
3 df['tip'] = imp_mean.fit_transform(df[['tip']])
4
5 # 2. SimpleImputer for categorical attributes (most
  frequent / mode)
6 imp_mode = SimpleImputer(strategy='most_frequent')
7 df['smoker'] = imp_mode.fit_transform(df[['smoker']]).
  ravel()
8
9 # 3. KNNImputer (multivariate k-nearest neighbors
  imputation)
10 knn_imp = KNNImputer(n_neighbors=2, weights='distance')
11 num_cols = ['total_bill', 'tip', 'size', 'age']
12 df[num_cols] = knn_imp.fit_transform(df[num_cols])

```

5.3 Multivariate Exploration and Correlation Analysis

5.3.1 Contingency Tables and Conditional Distributions

Evaluating dependencies across categorical features relies on row-normalized cross-tabulations:

```

1 # Joint frequency table between smoker status and gender
2 smoker_xt = pd.crosstab(df['smoker'], df['sex'])
3

```

```

4 # Row-wise normalization (proportional fractions)
5 smoker_pct = smoker_xt.div(smoker_xt.sum(1).astype(float)
6                             ), axis=0)
7 smoker_pct.plot(kind='bar', stacked=False, title='Gender
8               per Smoker Category')
9 plt.ylabel('Relative Frequency')
10 plt.show()

```

5.3.2 Correlation Matrices and Heatmap Rendering

Pandas computes Pearson, Spearman, and Kendall correlation matrices across numeric attributes:

```

1 # Pearson linear correlation matrix
2 corr_pearson = df.corr(numeric_only=True, method='
3     pearson')
4
5 # Spearman rank correlation matrix (robust monotonic
6     measure)
7 corr_spearman = df.corr(numeric_only=True, method='
8     spearman')
9
10 # Annotated heatmap rendering via Seaborn
11 plt.figure(figsize=(8, 6))
12 sns.heatmap(corr_pearson, annot=True, cmap='coolwarm',
13             fmt='.2f', vmin=-1, vmax=1)
14 plt.title('Pearson Correlation Heatmap (Tips Dataset)')
15 plt.show()

```

5.3.3 Scatter Matrix and Pairplots

Pairwise bivariate inspections are efficiently visualized using `scatter_matrix` or Seaborn `pairplot`:

```

1 # Pandas Scatter Matrix with KDE diagonal
2 pd.plotting.scatter_matrix(df[['total_bill', 'tip', '
3     size', 'age']], figsize=(9, 9), diagonal='kde')
4 plt.show()
5
6 # Seaborn Pairplot with class-based color coding
7 sns.pairplot(df, vars=['total_bill', 'tip', 'size'], hue
8               ='time', palette='Dark2')
9 plt.show()

```

5.3.4 Feature Engineering: Domain Ratio Construction

Domain expertise guides the derivation of novel features with enhanced explanatory power:

```
1 # 1. Effective tip ratio relative to total bill
2 df['tip_ratio'] = df['tip'] / df['total_bill']
3
4 # 2. Bill per person (BPP)
5 df['bpp'] = df['total_bill'] / df['size']
6
7 # Stratified Boxplot inspection across gender
8 df.boxplot(column=['bpp'], by='sex')
9 plt.title('Bill per Person Stratified by Gender')
10 plt.suptitle('')
11 plt.show()
```

5.4 Feature Transformation, Discretization, and Encoding

Remark 5.4.1 (Interactive Notebook Reference). Code examples in Sections 5.4 and 5.5 correspond to the interactive Jupyter [Notebook 2 \(Feature Engineering\)](#):

[2_feature_engineering_and_data_representation.ipynb](#)

5.4.1 Discretization with KBinsDiscretizer

Scikit-Learn's `KBinsDiscretizer` converts continuous attributes into discrete ordinal bins using three distinct binning strategies:

```
1 # Discretization into 4 bins across different strategies
2 disc_uniform = KBinsDiscretizer(n_bins=4, encode='
    ordinal', strategy='uniform')      # Equal-Width
3 disc_quantile = KBinsDiscretizer(n_bins=4, encode='
    ordinal', strategy='quantile')     # Equal-Frequency
4 disc_kmeans = KBinsDiscretizer(n_bins=4, encode='ordinal
    ', strategy='kmeans')              # Cluster-based
5
6 df['total_bill_bins'] = disc_uniform.fit_transform(df[['
    total_bill']])
7 print(df['total_bill_bins'].value_counts())
```

5.4.2 Feature Scaling: StandardScaler vs. MinMaxScaler

- **StandardScaler**: Standardizes features to zero mean and unit variance ($\mu = 0, \sigma = 1$).
- **MinMaxScaler**: Bounds values strictly into $[0, 1]$.

```
1 num_cols = ['total_bill', 'tip', 'age']
2
3 # Z-Score Standardization
4 scaler_std = StandardScaler()
5 df_standardized = df.copy()
6 df_standardized[num_cols] = scaler_std.fit_transform(df[
    num_cols])
7
8 # Min-Max Normalization
9 scaler_minmax = MinMaxScaler()
10 df_normalized = df.copy()
11 df_normalized[num_cols] = scaler_minmax.fit_transform(df[
    num_cols])
12
13 # Inverting transformation back to original scale
14 df_recovered = scaler_minmax.inverse_transform(
    df_normalized[num_cols])
```

5.4.3 Categorical Encoding: One-Hot vs. Target Encoding

```
1 # 1. One-Hot Encoding (prevents imposing synthetic order
2 )
3 ohe = OneHotEncoder(handle_unknown='ignore',
4     sparse_output=False)
5 encoded_days = ohe.fit_transform(df[['day']])
6 encoded_df = pd.DataFrame(encoded_days, columns=ohe.
7     get_feature_names_out(['day']))
8
9 # 2. Target Encoding (substitutes categories with target
10 expectation)
11 target_enc = TargetEncoder(smooth='auto')
12 # Encode 'day' using mean 'tip'
```

```

9 df['day_target_encoded'] = target_enc.fit_transform(df[['
    'day']], df['tip'])

```

5.5 Dimensionality Reduction: PCA and *t*-SNE

5.5.1 Principal Component Analysis (PCA)

Features must be centered and scaled before fitting PCA so that high-variance variables do not arbitrarily dominate the spectral decomposition:

```

1 # Select quantitative predictors
2 X = df[['total_bill', 'tip', 'size', 'age', 'tip_ratio',
    'bpp']].dropna()
3 X_scaled = StandardScaler().fit_transform(X)
4
5 # Fit PCA
6 pca = PCA()
7 X_pca = pca.fit_transform(X_scaled)
8
9 # Eigenvalues (absolute variance explained per component
    )
10 eigenvalues = pca.explained_variance_
11 # Relative variance ratio
12 explained_var_ratio = pca.explained_variance_ratio_
13 # Cumulative explained variance
14 cumulative_var = np.cumsum(explained_var_ratio)

```

Scree Plot Visualization

The Scree Plot helps detect the characteristic "elbow" where incremental principal components contribute negligible variance:

```

1 plt.figure(figsize=(8, 4))
2 plt.plot(range(1, len(eigenvalues) + 1), eigenvalues, 'o
    -', color='navy', label='Individual Eigenvalues')
3 plt.title('PCA Scree Plot (Eigenvalues)')
4 plt.xlabel('Principal Component Index')
5 plt.ylabel('Eigenvalue (Explained Variance)')
6 plt.grid(True, linestyle='--', alpha=0.5)
7 plt.legend()
8 plt.show()

```

5.5.2 Non-linear Projection via *t*-SNE

t-Distributed Stochastic Neighbor Embedding (*t*-SNE) is an unsupervised non-linear algorithm designed to preserve local neighborhood affinities in high-dimensional datasets:

```
1 tsne = TSNE(  
2     n_components=2,  
3     perplexity=30,          # Balance between local and  
4                               global structure (typical range: 5-50)  
5     learning_rate=200,      # Gradient descent step size  
6     max_iter=1000,          # Maximum KL-divergence  
7                               optimization iterations  
8     random_state=42  
9 )  
10 X_tsne = tsne.fit_transform(X_scaled)
```

5.5.3 Methodological Comparison: PCA vs. *t*-SNE

```
1 # Side-by-side comparative scatter plots  
2 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))  
3  
4 # PCA projection  
5 ax1.scatter(X_pca[:, 0], X_pca[:, 1], c='navy', alpha  
6             =0.7, edgecolors='k')  
7 ax1.set_title('Linear PCA Projection (PC1 vs PC2)')  
8 ax1.set_xlabel('First Principal Component')  
9 ax1.set_ylabel('Second Principal Component')  
10 ax1.grid(True, linestyle='--', alpha=0.5)  
11  
12 # t-SNE projection  
13 ax2.scatter(X_tsne[:, 0], X_tsne[:, 1], c='crimson',  
14            alpha=0.7, edgecolors='k')  
15 ax2.set_title('Non-linear t-SNE Projection')  
16 ax2.set_xlabel('t-SNE Dimension 1')  
17 ax2.set_ylabel('t-SNE Dimension 2')  
18 ax2.grid(True, linestyle='--', alpha=0.5)  
19  
20 plt.tight_layout()  
21 plt.show()
```

Criterion	PCA (Principal Component Analysis)	t -SNE (t -Distributed SNE)
Nature	Linear, global, deterministic.	Non-linear, local, stochastic/probabilistic.
Objective	Maximizes projected global variance via eigenvalue decomposition.	Minimizes Kullback-Leibler divergence between neighborhood probability distributions.
Interpretability	Component axes are linear feature combinations with explicit loadings.	Axes possess no absolute physical meaning; only relative neighbor proximity matters.
Long-Range Distances	Preserves large-scale Euclidean spatial configurations.	Distorts global inter-cluster distances (cluster radii do not represent true cluster volume).
Standard Use	Dimensionality reduction for supervised learning, compression, decorrelation.	Exploratory manifold visualization and qualitative cluster identification (2D/3D).

Table 5.1: Comparative synthesis of PCA and t -SNE characteristics.

Chapter 6

Proximity Measures, Information Theory, and Cluster Analysis

This chapter establishes the computational and mathematical foundations underpinning unsupervised learning and instance-based reasoning in Data Mining:

1. Analytical formulation of **proximity, distance, and similarity** across continuous, discrete, and heterogeneous attributes;
2. Formal tools from **Shannon’s Information Theory** (entropy and mutual information) for non-linear dependency and uncertainty quantification;
3. Core principles, taxonomies, and geometric formulations of **Cluster Analysis**.

6.1 Foundations of Similarity, Dissimilarity, and Proximity

Across Data Mining, Machine Learning, and Information Retrieval, pairwise comparisons form the algorithmic backbone of numerous paradigms: from nearest neighbor classification (k -NN) to cluster discovery, density estimation, and outlier detection.

Definition 6.1.1 (Similarity Measure). A **similarity measure** is a real-valued function $s : \mathcal{D} \times \mathcal{D} \rightarrow \mathbb{R}$ quantifying the degree of affinity between two objects $\mathbf{x}$ and $\mathbf{y}$. This value increases monotonically as the objects grow closer: it reaches its upper bound when the two objects are identical and

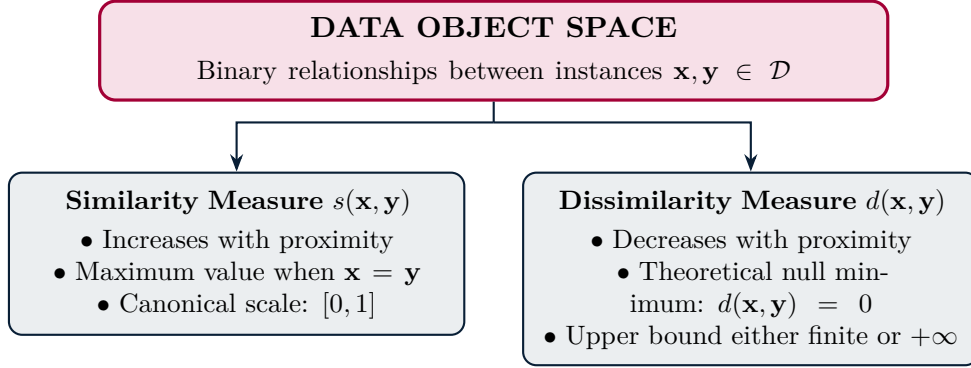


Figure 6.1: Conceptual hierarchy of proximity measures across the data domain.

decreases as they diverge. In most settings, it is normalized to $[0, 1]$, where 1 denotes total identity and 0 complete disjunction.

Definition 6.1.2 (Dissimilarity Measure). A **dissimilarity measure** is a real-valued function $d : \mathcal{D} \times \mathcal{D} \rightarrow \mathbb{R}$ quantifying the discrepancy between two objects $\mathbf{x}$ and $\mathbf{y}$. It assumes lower values when objects are affine or identical, increasing monotonically as their difference widens. The lower bound is conventionally $d(\mathbf{x}, \mathbf{y}) = 0$ when $\mathbf{x} = \mathbf{y}$, whereas the upper bound may be bounded (1) or unbounded ($+\infty$).

The umbrella term **proximity** is used across the literature to refer interchangeably to either a similarity or a dissimilarity measure.

6.2 Single-Attribute Proximity Measures

Given two objects $\mathbf{x}$ and $\mathbf{y}$, let p be the scalar value of $\mathbf{x}$ along an attribute and q the value of $\mathbf{y}$ along the same attribute. The derivation of elementary dissimilarity $d(p, q)$ and dual similarity $s(p, q)$ depends on the attribute's measurement scale.

6.2.1 Nominal Attributes

Because qualitative nominal categories (e.g., color, department name) exhibit neither natural ordering nor metric properties, equality and inequality evaluations are the sole permissible operators:

$$d(p, q) = \begin{cases} 0 & \text{if } p = q \\ 1 & \text{if } p \neq q \end{cases}, \quad s(p, q) = \begin{cases} 1 & \text{if } p = q \\ 0 & \text{if } p \neq q \end{cases} \quad (6.1)$$

Attribute Scale	Dissimilarity $d(p, q)$	Dual Similarity $s(p, q)$
Nominal	$d = 0$ if $p = q$; $d = 1$ if $p \neq q$	$s = 1$ if $p = q$; $s = 0$ if $p \neq q$
Ordinal	$d = \frac{ p - q }{n - 1}$ (with states mapped to $\{0, \dots, n - 1\}$)	$s = 1 - d = 1 - \frac{ p - q }{n - 1}$
Interval or Ratio	$d = p - q $	$s = -d, \quad s = \frac{1}{1 + d},$ or $s = 1 - \frac{d - d_{\min}}{d_{\max} - d_{\min}}$

Table 6.1: Formulation rules for elementary proximity across single attribute types.

6.2.2 Ordinal Attributes

Ordinal attributes possess an intrinsic rank composed of n distinct states. To obtain consistent distance values, discrete categories are mapped to positional integers $\{0, 1, \dots, n - 1\}$.

The absolute difference $|p - q|$ ranges from 0 to $n - 1$. Dividing by the maximum observable spread guarantees normalization to $[0, 1]$:

$$d(p, q) = \frac{|p - q|}{n - 1}, \quad s(p, q) = 1 - d(p, q) = 1 - \frac{|p - q|}{n - 1} \quad (6.2)$$

Example 6.2.1 (Similarity Between Ordinal Ratings). Consider an academic performance ranking over $n = 5$ states:

$$\{\text{fail} = 0, \text{pass} = 1, \text{good} = 2, \text{very good} = 3, \text{excellent} = 4\} \quad (6.3)$$

For two candidates with ratings $p = 1$ (*pass*) and $q = 4$ (*excellent*):

$$d(1, 4) = \frac{|1 - 4|}{5 - 1} = \frac{3}{4} = 0.75 \implies s(1, 4) = 1 - 0.75 = 0.25 \quad (6.4)$$

6.2.3 Interval and Ratio Attributes

For continuous or discrete numeric variables, the elementary distance is the absolute difference $d(p, q) = |p - q|$. Converting to bounded similarity $[0, 1]$ follows three main formulations:

1. **Opposite Translation** (unbounded above): $s(p, q) = -|p - q|$;

2. **Asymptotic Inversion:** $s(p, q) = \frac{1}{1 + |p - q|}$, yielding $s \rightarrow 1$ as $d \rightarrow 0$ and $s \rightarrow 0$ as $d \rightarrow \infty$;
3. **Linear Min-Max Rescaling:** $s(p, q) = 1 - \frac{d(p, q) - d_{\min}}{d_{\max} - d_{\min}}$, where $d_{\min} = 0$ and $d_{\max}$ represents the maximum observed attribute range.

6.3 Vector Distances and Minkowski Metrics

When objects are described by multidimensional continuous vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, overall dissimilarity aggregates dimensional discrepancies across all n orthogonal coordinates.

Definition 6.3.1 (Minkowski Distance (L_r Norm)). Given two vectors $\mathbf{x} = (x_1, \dots, x_n)$ and $\mathbf{y} = (y_1, \dots, y_n)$ in $\mathbb{R}^n$, the **Minkowski distance** with parameter $r \geq 1$ is defined as:

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_r = \left(\sum_{k=1}^n |x_k - y_k|^r \right)^{1/r} \quad (6.5)$$

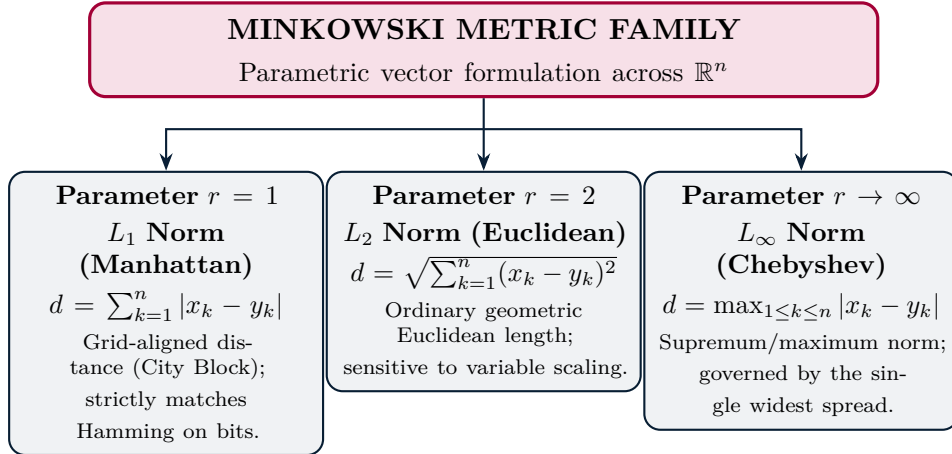


Figure 6.2: Structure of the Minkowski metric family across variations of parameter r .

1. **Manhattan Distance** ($r = 1$, L_1 Norm, City Block):

$$d_{L_1}(\mathbf{x}, \mathbf{y}) = \sum_{k=1}^n |x_k - y_k| \quad (6.6)$$

Measures rectilinear paths along orthogonal coordinate axes. On binary indicator vectors, it is strictly equivalent to the **Hamming distance**.

2. **Euclidean Distance** ($r = 2$, L_2 **Norm**):

$$d_{L_2}(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^n (x_k - y_k)^2} \quad (6.7)$$

Corresponds to direct geometric straight-line length. Standardizing attributes via z -scores is mandatory when features possess disparate measurement units to avoid dominance by large variance dimensions.

3. **Supremum Distance** ($r \rightarrow \infty$, L_∞ **Norm**, **Chebyshev Distance**):

$$d_{L_\infty}(\mathbf{x}, \mathbf{y}) = \lim_{r \rightarrow \infty} \left(\sum_{k=1}^n |x_k - y_k|^r \right)^{1/r} = \max_{1 \leq k \leq n} |x_k - y_k| \quad (6.8)$$

Isolates the single largest coordinate deviation between the pair of points.

6.4 Worked Example of Distance Matrix Computation

Consider the set of 4 two-dimensional points evaluated in lecture:

$$\mathbf{p}_1 = (0, 2), \quad \mathbf{p}_2 = (2, 0), \quad \mathbf{p}_3 = (3, 1), \quad \mathbf{p}_4 = (5, 1) \quad (6.9)$$

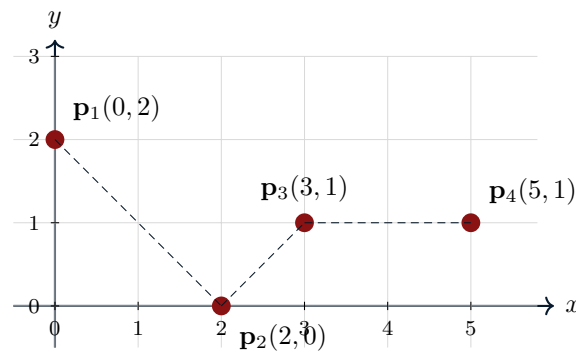


Figure 6.3: Geometric placement of four sample data points on the Cartesian plane.

Evaluating analytical distances across distinct pairs yields:

- $(\mathbf{p}_1, \mathbf{p}_2)$: $\Delta x = 2, \Delta y = 2 \implies L_1 = 4, L_2 = \sqrt{8} \approx 2.828, L_\infty = 2$;
- $(\mathbf{p}_1, \mathbf{p}_3)$: $\Delta x = 3, \Delta y = 1 \implies L_1 = 4, L_2 = \sqrt{10} \approx 3.162, L_\infty = 3$;
- $(\mathbf{p}_1, \mathbf{p}_4)$: $\Delta x = 5, \Delta y = 1 \implies L_1 = 6, L_2 = \sqrt{26} \approx 5.099, L_\infty = 5$;
- $(\mathbf{p}_2, \mathbf{p}_3)$: $\Delta x = 1, \Delta y = 1 \implies L_1 = 2, L_2 = \sqrt{2} \approx 1.414, L_\infty = 1$;
- $(\mathbf{p}_2, \mathbf{p}_4)$: $\Delta x = 3, \Delta y = 1 \implies L_1 = 4, L_2 = \sqrt{10} \approx 3.162, L_\infty = 3$;
- $(\mathbf{p}_3, \mathbf{p}_4)$: $\Delta x = 2, \Delta y = 0 \implies L_1 = 2, L_2 = \sqrt{4} = 2.000, L_\infty = 2$.

Since distance metrics enforce reflexivity ($d(\mathbf{p}_i, \mathbf{p}_i) = 0$) and symmetry ($d(\mathbf{p}_i, \mathbf{p}_j) = d(\mathbf{p}_j, \mathbf{p}_i)$), the complete distance matrices are symmetric with zero main diagonals:

$$\mathbf{D}_{L_1} = \begin{bmatrix} 0 & 4 & 4 & 6 \\ 4 & 0 & 2 & 4 \\ 4 & 2 & 0 & 2 \\ 6 & 4 & 2 & 0 \end{bmatrix}, \quad \mathbf{D}_{L_\infty} = \begin{bmatrix} 0 & 2 & 3 & 5 \\ 2 & 0 & 1 & 3 \\ 3 & 1 & 0 & 2 \\ 5 & 3 & 2 & 0 \end{bmatrix}, \quad (6.10)$$

$$\mathbf{D}_{L_2} = \begin{bmatrix} 0 & 2.828 & 3.162 & 5.099 \\ 2.828 & 0 & 1.414 & 3.162 \\ 3.162 & 1.414 & 0 & 2.000 \\ 5.099 & 3.162 & 2.000 & 0 \end{bmatrix} \quad (6.11)$$

6.5 Metric Axioms and Mathematical Properties

Definition 6.5.1 (Metric Distance Axioms). A dissimilarity function $d : \mathcal{D} \times \mathcal{D} \rightarrow \mathbb{R}$ qualifies as a formal **metric** if and only if it satisfies three fundamental axioms for all triplets $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathcal{D}$:

1. **Positive Definiteness:**

$$d(\mathbf{x}, \mathbf{y}) \geq 0 \quad \forall \mathbf{x}, \mathbf{y}; \quad d(\mathbf{x}, \mathbf{y}) = 0 \iff \mathbf{x} = \mathbf{y} \quad (6.12)$$

2. **Symmetry:**

$$d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x}) \quad \forall \mathbf{x}, \mathbf{y} \quad (6.13)$$

3. **Triangle Inequality:**

$$d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z}) \quad \forall \mathbf{x}, \mathbf{y}, \mathbf{z} \quad (6.14)$$

All Minkowski distance metrics ($r \geq 1$) strictly satisfy these three conditions (with the triangle inequality arising from Minkowski's inequality for sums and integrals).

Similarly, canonical similarity measures $s : \mathcal{D} \times \mathcal{D} \rightarrow \mathbb{R}$ satisfy:

1. **Maximum Identity:** $s(\mathbf{x}, \mathbf{y}) = 1$ (or maximum theoretical upper bound) if and only if $\mathbf{x} = \mathbf{y}$;
2. **Symmetry:** $s(\mathbf{x}, \mathbf{y}) = s(\mathbf{y}, \mathbf{x})$ for all pairs $\mathbf{x}, \mathbf{y}$.

6.6 Binary Proximity Measures: SMC vs Jaccard Index

In feature spaces spanned by asymmetric binary attributes (e.g., transaction baskets or vocabulary occurrence in text retrieval), evaluating concordance requires distinguishing between attribute presence (1) and absence (0).

For two binary vectors $\mathbf{p}$ and $\mathbf{q}$, pairwise state occurrences populate the 2×2 contingency table:

		Object $\mathbf{q}$	
		1	0
Object $\mathbf{p}$	1	M_{11} (joint presence)	M_{10} (present only in $\mathbf{p}$)
	0	M_{01} (present only in $\mathbf{q}$)	M_{00} (joint absence)

Table 6.2: Contingency table for binary attribute vectors.

The total number of binary attributes equals:

$$n = M_{11} + M_{10} + M_{01} + M_{00} \quad (6.15)$$

6.6.1 Coefficient Formulations

1. **Simple Matching Coefficient (SMC):**

$$\text{SMC} = \frac{M_{11} + M_{00}}{M_{01} + M_{10} + M_{11} + M_{00}} \quad (6.16)$$

Weights positive agreements (M_{11}) and negative agreements (M_{00}) equally. It is appropriate for symmetric binary features (such as biological sex or balanced binary survey indicators).

2. **Jaccard Index (J):**

$$J = \frac{M_{11}}{M_{01} + M_{10} + M_{11}} \quad (6.17)$$

Treats the binary feature as asymmetric: deliberately ignores joint absences (M_{00}).

Why Jaccard is Essential for Sparse Data

In high-dimensional and sparse domains (such as market basket analysis or text mining), any two records jointly lack thousands of items or terms. Using the SMC coefficient would cause M_{00} to dominate the fraction numerically, creating artificially inflated similarity ($\text{SMC} \rightarrow 1$) simply because both transactions did not purchase the vast majority of catalog products. Jaccard avoids this distortion by restricting evaluation strictly to active items.

Example 6.6.1 (SMC vs Jaccard Comparison on Sparse Vectors). Given two binary vectors with $n = 10$ features:

$$\mathbf{p} = [1, 0, 0, 0, 0, 0, 0, 0, 0, 0] \quad (6.18)$$

$$\mathbf{q} = [0, 0, 0, 0, 0, 0, 1, 0, 0, 1] \quad (6.19)$$

Inspecting indices yields:

$$M_{11} = 0, \quad M_{10} = 1, \quad M_{01} = 2, \quad M_{00} = 7 \quad (6.20)$$

The coefficients evaluate to:

$$\text{SMC} = \frac{0 + 7}{2 + 1 + 0 + 7} = \frac{7}{10} = 0.70, \quad J = \frac{0}{2 + 1 + 0} = \frac{0}{3} = 0.00 \quad (6.21)$$

SMC indicates 70% affinity based entirely on shared non-occurrences, while Jaccard indicates 0% similarity since the two items share no active components.

6.7 Cosine Similarity for Document Data

In the Vector Space Model of Information Retrieval, text documents are represented as high-dimensional term frequency or TF-IDF vectors.

In this context, documents of differing lengths covering identical subject matter appear distant under the Euclidean metric due to differing vector magnitudes. **Cosine Similarity** captures directional alignment regardless of vector norm:

$$\cos(\mathbf{d}_1, \mathbf{d}_2) = \frac{\mathbf{d}_1 \cdot \mathbf{d}_2}{\|\mathbf{d}_1\|_2 \|\mathbf{d}_2\|_2} = \frac{\sum_{k=1}^n d_{1,k} d_{2,k}}{\sqrt{\sum_{k=1}^n d_{1,k}^2} \sqrt{\sum_{k=1}^n d_{2,k}^2}} \quad (6.22)$$

Example 6.7.1 (Worked Example on Document Term Vectors). Given two term count vectors:

$$\mathbf{d}_1 = [3, 2, 0, 5, 0, 0, 0, 2, 0, 0] \quad (6.23)$$

$$\mathbf{d}_2 = [1, 0, 0, 0, 0, 0, 0, 1, 0, 2] \quad (6.24)$$

1. **Dot Product:**

$$\mathbf{d}_1 \cdot \mathbf{d}_2 = (3 \times 1) + (2 \times 0) + (5 \times 0) + (2 \times 1) + (0 \times 2) = 3 + 2 = 5 \quad (6.25)$$

2. **Euclidean Norms:**

$$\|\mathbf{d}_1\|_2 = \sqrt{3^2 + 2^2 + 5^2 + 2^2} = \sqrt{9 + 4 + 25 + 4} = \sqrt{42} \approx 6.4807 \quad (6.26)$$

$$\|\mathbf{d}_2\|_2 = \sqrt{1^2 + 1^2 + 2^2} = \sqrt{1 + 1 + 4} = \sqrt{6} \approx 2.4495 \quad (6.27)$$

3. **Cosine Similarity:**

$$\cos(\mathbf{d}_1, \mathbf{d}_2) = \frac{5}{\sqrt{42} \times \sqrt{6}} = \frac{5}{\sqrt{252}} = \frac{5}{15.8745} \approx 0.3150 \quad (6.28)$$

The documents share roughly 31.5% angular thematic alignment.

6.8 Weighting, Missing Values, and Correlation Across Objects

6.8.1 Weighted Similarity with Missing Data Handling

When attributes carry uneven importance or contain null values, non-negative weights $\{\omega_k\}_{k=1}^n$ and a binary indicator variable $\delta_k \in \{0, 1\}$ are incorporated ($\delta_k = 1$ if and only if feature k is present in both $\mathbf{x}$ and $\mathbf{y}$):

$$\text{similarity}(\mathbf{x}, \mathbf{y}) = \frac{\sum_{k=1}^n \omega_k \delta_k s_k(\mathbf{x}, \mathbf{y})}{\sum_{k=1}^n \omega_k \delta_k} \quad (6.29)$$

Analogously, the **weighted Minkowski distance** is formulated as:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^n \omega_k |x_k - y_k|^r \right)^{1/r} \quad (6.30)$$

6.8.2 Correlation Between Objects

Pearson's correlation coefficient can be calculated across rows to measure affinity between object profiles $\mathbf{p}$ and $\mathbf{q}$:

$$\text{corr}(\mathbf{p}, \mathbf{q}) = \frac{\sum_{k=1}^n (p_k - \bar{p})(q_k - \bar{q})}{\sqrt{\sum_{k=1}^n (p_k - \bar{p})^2} \sqrt{\sum_{k=1}^n (q_k - \bar{q})^2}} = \mathbf{p}^* \cdot \mathbf{q}^* \quad (6.31)$$

where $\mathbf{p}^*$ and $\mathbf{q}^*$ denote standardized profiles with zero mean and unit variance.

6.9 Information Theory: Entropy and Mutual Information

Information Theory, pioneered by Claude Shannon (1948), supplies the mathematical foundation for evaluating disorder, uncertainty, and non-linear dependencies among discrete or binned variables.

6.9.1 Self-Information and Shannon Entropy

The information content associated with an event x_i occurring with probability $p_i = P(X = x_i)$ represents surprise:

$$I(x_i) = \log_2 \left(\frac{1}{p_i} \right) = -\log_2(p_i) \quad (6.32)$$

Shannon Entropy $H(X)$ represents the expected value of self-information:

$$H(X) = \mathbb{E}[I(X)] = -\sum_{i=1}^n p_i \log_2(p_i) \quad (6.33)$$

with the continuity convention $\lim_{p \rightarrow 0^+} p \log_2(p) = 0$.

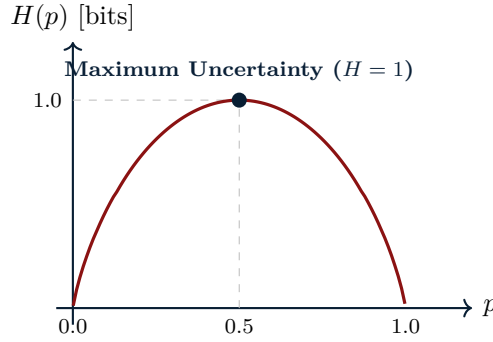


Figure 6.4: Shannon entropy curve for a Bernoulli random variable as a function of parameter p .

Entropy is strictly bounded: $0 \leq H(X) \leq \log_2(n)$. It vanishes if and only if the event is deterministic ($p_k = 1$), reaching its maximum $\log_2(n)$ under a uniform distribution ($p_i = 1/n$).

6.9.2 Mutual Information

Mutual Information $I(X; Y)$ quantifies shared information between variables (the reduction in uncertainty regarding X resulting from knowledge of

Y):

$$I(X; Y) = H(X) + H(Y) - H(X, Y) \quad (6.34)$$

where $H(X, Y)$ is the joint entropy over joint probabilities $p_{ij} = P(X = x_i, Y = y_j)$:

$$H(X, Y) = - \sum_{i=1}^{n_X} \sum_{j=1}^{n_Y} p_{ij} \log_2(p_{ij}) \quad (6.35)$$

Fundamental properties:

1. $I(X; Y) \geq 0$ and $I(X; Y) = I(Y; X)$ (non-negativity and symmetry);
2. $I(X; Y) = 0 \iff P(X, Y) = P(X)P(Y)$ (vanishes under statistical independence);
3. $I(X; Y) \leq \min(H(X), H(Y)) \leq \log_2(\min(n_X, n_Y))$.

6.9.3 Worked Calculation Example of Mutual Information

Consider the sample of $m = 100$ students evaluated in lecture, comparing **Academic Status** ($X \in \{\text{Undergrad}, \text{Grad}\}$) with **Grade** earned ($Y \in \{A, B, C\}$).

Status (X)	Grade A	Grade B	Grade C	Total (X)
Undergrad	5 (0.05)	30 (0.30)	10 (0.10)	45 (0.45)
Grad	30 (0.30)	20 (0.20)	5 (0.05)	55 (0.55)
Total (Y)	35 (0.35)	50 (0.50)	15 (0.15)	100 (1.00)

Table 6.3: Joint and marginal frequency distributions for academic status and course grades.

1. Marginal Entropy of X :

$$H(X) = -0.45 \log_2(0.45) - 0.55 \log_2(0.55) \approx 0.9928 \text{ bits} \quad (6.36)$$

2. Marginal Entropy of Y :

$$\begin{aligned} H(Y) &= -0.35 \log_2(0.35) - 0.50 \log_2(0.50) - 0.15 \log_2(0.15) \\ &\approx 0.5301 + 0.5000 + 0.4105 = 1.4406 \text{ bits} \end{aligned} \quad (6.37)$$

3. **Joint Entropy** $H(X, Y)$:

$$\begin{aligned}
 H(X, Y) &= - \sum_{i=1}^2 \sum_{j=1}^3 p_{ij} \log_2(p_{ij}) \\
 &\approx 2(0.2161) + 2(0.5211) + 0.3322 + 0.4644 = 2.2710 \text{ bits}
 \end{aligned} \tag{6.38}$$

4. **Mutual Information**:

$$\begin{aligned}
 I(X; Y) &= H(X) + H(Y) - H(X, Y) \\
 &= 0.9928 + 1.4406 - 2.2710 = 0.1624 \text{ bits}
 \end{aligned} \tag{6.39}$$

Knowledge of the student's academic status reduces the uncertainty regarding their grade distribution by 0.1624 bits (accounting for 11.27% of the baseline grade entropy).

6.10 Foundations and Objectives of Cluster Analysis

Cluster Analysis (unsupervised classification) partitions an unlabelled dataset $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ into homogeneous groups (**clusters**) according to two dual guiding principles:

1. **Maximizing Intra-Cluster Cohesion**: Objects assigned to the same cluster must share high reciprocal affinity and proximity;
2. **Maximizing Inter-Cluster Separation**: Objects located across different clusters must exhibit maximum metric distance.

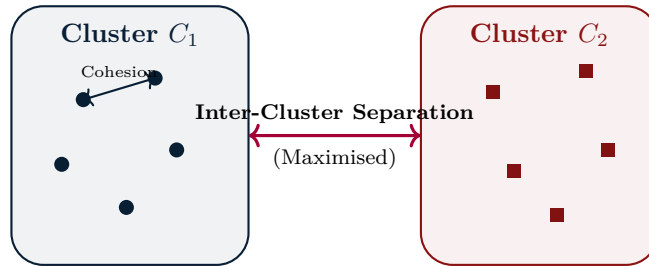


Figure 6.5: Geometric trade-off between internal cohesion and external separation in cluster analysis.

Unlike supervised learning, no target labels y are provided; structural boundaries are derived organically from sample geometry.

6.10.1 Applications: Understanding vs Summarization

Practical motivations for clustering fall into two complementary domains:

1. **Data Understanding:** Uncovering latent taxonomic patterns:
 - *Information Retrieval:* Hierarchical structuring of unstructured document libraries;
 - *Bioinformatics:* Grouping genes exhibiting synchronous functional co-expression profiles;
 - *Financial Markets:* Identifying co-moving equities (e.g., tech, energy, or banking clusters).
2. **Data Summarization:** Quantitative data compression where voluminous observations are condensed into representative prototypes (centroids or medoids), conserving aggregate statistical characteristics.

6.10.2 What Cluster Analysis is NOT

To maintain clear boundaries, clustering must be distinguished from related data tasks:

- **Manual Partitioning:** Arbitrary external assignment (e.g., grouping by name initial $A - L$ vs $M - Z$) reflects no natural data structure;
- **Database Querying:** SQL filter clauses (e.g., `WHERE Age > 30`) partition data by user-defined constraints rather than intrinsic topology;
- **Supervised Classification:** Optimizes explicit prediction against known target labels $y \in \mathcal{Y}$;
- **Pattern Mining:** Association rules discover local item implications across transactions, whereas clustering determines global structure across full feature vectors.

6.11 Clustering Taxonomies and Grouping Structures

6.11.1 Partitional vs Hierarchical Clustering

1. **Partitional Clustering:** Divides space into k disjoint subsets $\mathcal{C} = \{C_1, \dots, C_k\}$ that fully cover the dataset:

$$\bigcup_{i=1}^k C_i = \mathcal{D}, \quad C_i \cap C_j = \emptyset \quad \forall i \neq j \quad (6.40)$$

Each object belongs to exactly one cluster, requiring k to be specified upfront (e.g., K -Means).

2. **Hierarchical Clustering:** Produces a nested family of partitions organized as a metric tree called a **dendrogram**, eliminating the need to set k in advance.

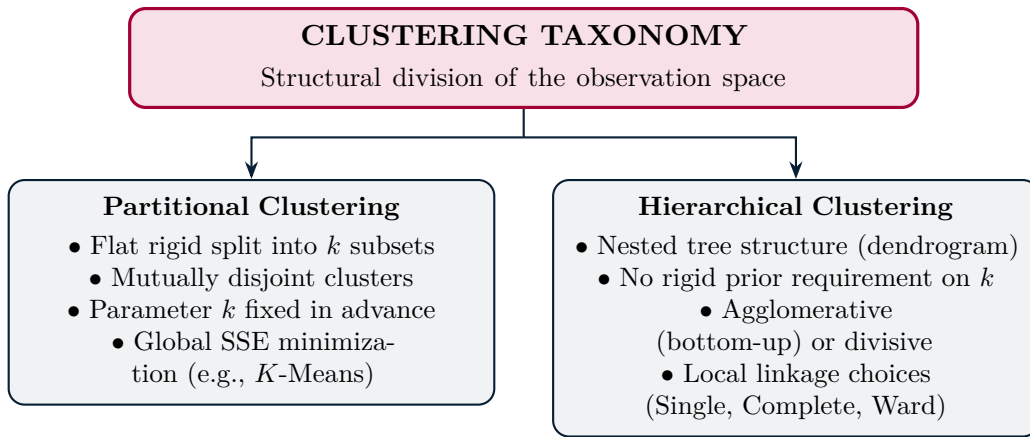


Figure 6.6: Structural comparison between partitional and hierarchical clustering.

6.11.2 Additional Structural Dimensions

- **Exclusive vs Non-Exclusive (Overlapping):** In non-exclusive clustering, points may belong to multiple groups simultaneously (ideal for border observations or multi-topic texts).
- **Fuzzy (Soft) vs Non-Fuzzy (Hard):** In fuzzy clustering, each instance $\mathbf{x}_i$ is assigned a continuous membership degree $w_{ik} \in [0, 1]$ satisfying $\sum_{k=1}^K w_{ik} = 1$.
- **Partial vs Complete:** Partial methods identify dense cohesive clusters while filtering out background noise and anomalies.
- **Homogeneous vs Heterogeneous:** Homogeneous algorithms presume similar geometric shapes and densities, whereas heterogeneous models accommodate arbitrary geometries and varying densities.

6.12 Cluster Types and Mathematical Formulations

The definition of a “cluster” differs significantly depending on the governing geometric concept:

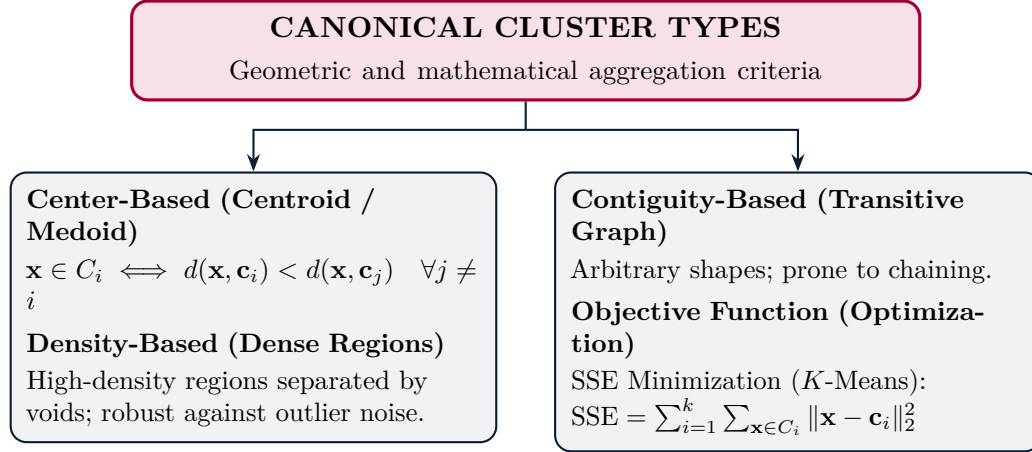


Figure 6.7: Overview of mathematical cluster formulations.

1. **Well-Separated Clusters:** Every point in a cluster is strictly closer to all cluster members than to any outside observation:

$$\forall \mathbf{x}, \mathbf{y} \in C_i, \forall \mathbf{z} \notin C_i \implies d(\mathbf{x}, \mathbf{y}) < d(\mathbf{x}, \mathbf{z}) \quad (6.41)$$

2. **Center-Based Clusters:** An object belongs to the cluster with the nearest prototype center:

$$\mathbf{x} \in C_i \iff d(\mathbf{x}, \mathbf{c}_i) < d(\mathbf{x}, \mathbf{c}_j) \quad \forall j \neq i \quad (6.42)$$

The center can be the **centroid** (continuous arithmetic mean $\mathbf{c}_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$) or the **medoid** (most central real instance $\mathbf{m}_i = \arg \min_{\mathbf{y} \in C_i} \sum_{\mathbf{x} \in C_i} d(\mathbf{x}, \mathbf{y})$).

3. **Contiguity-Based (Graph-Based) Clusters:** A point belongs to a cluster if it is closer to at least one cluster member than to any outside observation:

$$\forall \mathbf{x} \in C_i, \exists \mathbf{y} \in C_i \setminus \{\mathbf{x}\} \quad \text{s.t.} \quad d(\mathbf{x}, \mathbf{y}) < \min_{\mathbf{z} \notin C_i} d(\mathbf{x}, \mathbf{z}) \quad (6.43)$$

Accommodates arbitrary non-convex geometries, but suffers from the **chaining effect**, where isolated noise observations bridge and improperly merge distinct groups.

4. **Density-Based Clusters:** Dense regions bounded and separated by regions of low point density (e.g., DBSCAN). Resilient to noise and unconstrained by spherical geometry.
5. **Objective-Function-Defined Clusters:** Framed as optimization problems. Exhaustive global discovery across N points and k clusters scales with the Stirling numbers of the second kind:

$$S(N, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^N \approx \frac{k^N}{k!} \quad (6.44)$$

As the problem is inherently **NP-Hard**, iterative descent heuristics such as K -Means minimize the Sum of Squared Errors (SSE):

$$\text{SSE} = \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{c}_i\|_2^2 \quad (6.45)$$

6.13 Input Data Characteristics and Algorithm Families

Clustering effectiveness depends on input space characteristics:

- **Curse of Dimensionality:** In high-dimensional spaces $\mathbb{R}^n$, pairwise distances concentrate and equalize, diminishing the contrast between near and far neighbors;
- **Sparsity and Heterogeneity:** Sparse vectors require asymmetric metrics (Jaccard, Cosine); mixed data types require hybrid metrics (Gower);
- **Noise and Outliers:** Distort prototype centroids in center-based methods and trigger spurious bridging in contiguity-based methods.

The three principal algorithm families are structured as follows:

1. **K -Means and Variants:** Partitional, exclusive, complete. Iteratively minimizes SSE across compact spherical clusters of comparable density;
2. **Agglomerative Hierarchical Clustering:** Hierarchical, nested. Iteratively merges closest cluster pairs according to linkage criteria (Single, Complete, Average, Ward), producing an interpretable dendrogram;
3. **Density-Based Clustering (DBSCAN):** Local density-based, partial, non-parametric with respect to k . Naturally isolates noise points and detects clusters of arbitrary shape.

Chapter 7

Cluster Analysis: K-Means and Hierarchical Clustering Algorithms

In this chapter, we present the theoretical foundations, algebraic formulations, and operational dynamics of the two fundamental paradigms of unsupervised learning based on partitioning and hierarchy building:

1. The **K-Means Algorithm**: analytical formulation, Lloyd’s dynamics, minimization of the sum of squared errors (SSE), formal proof of convergence via Coordinate Descent, initialization heuristics (K-Means++, Bisecting K-Means), and structural limitations;
2. **Hierarchical Clustering**: agglomerative and divisive approaches, dendrogram interpretation, inter-cluster linkage criteria (MIN, MAX, Group Average, Ward’s method with analytical derivation), universal Lance-Williams recurrence relation, and graph-theoretic equivalence with the Minimum Spanning Tree (MST).

7.1 Introduction to Partitional and Hierarchical Cluster Analysis

Cluster Analysis (or unsupervised grouping) constitutes one of the foundational pillars of unsupervised machine learning. Unlike supervised learning, where each instance $\mathbf{x}_i$ is accompanied by a ground-truth class label $y_i \in \mathcal{Y}$ known a priori, the primary objective of clustering is to discover latent structures, geometric patterns, or natural groupings within an unlabelled multidimensional dataset.

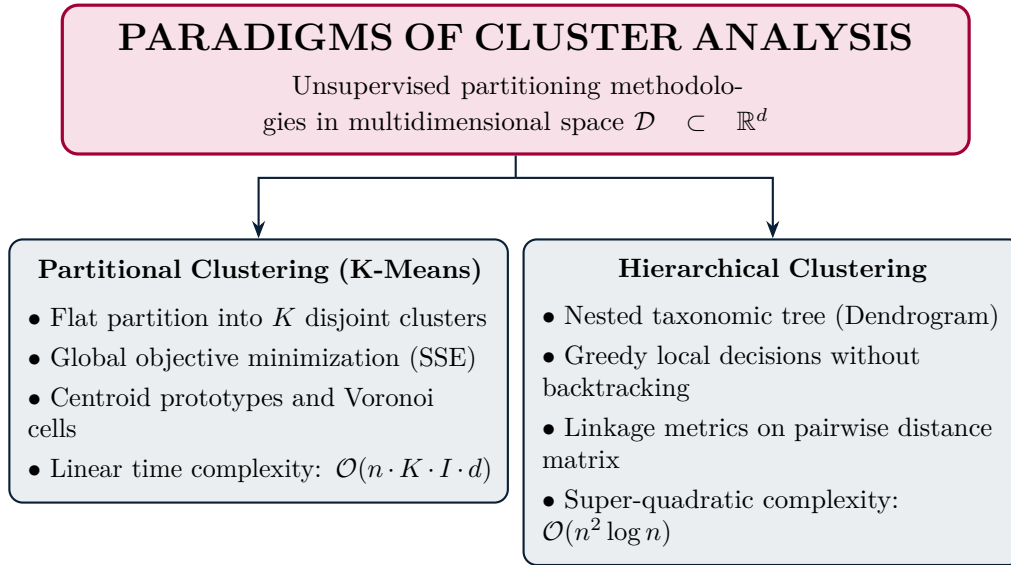


Figure 7.1: Taxonomy of the two core clustering paradigms covered in this chapter.

Given a dataset of n observations $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ with $\mathbf{x}_i \in \mathbb{R}^d$, the general clustering task requires partitioning $\mathcal{D}$ into disjoint subsets $C = \{C_1, \dots, C_K\}$ while simultaneously maximizing two complementary properties:

1. **High intra-cluster cohesion:** observations assigned to the same group must be as close and similar to one another as possible under the chosen metric.
2. **High inter-cluster separation:** items belonging to distinct groups must exhibit marked distance or dissimilarity from one another.

The two paradigms tackle this objective with fundamentally distinct philosophies: partitional clustering fixes the number of groups K a priori and pursues an optimal polygonal tessellation of space via prototypes; hierarchical clustering generates a nested sequence of partitions, deferring the choice of K to post-hoc dendrogram inspection.

7.2 K-Means: Lloyd’s Algorithm and Voronoi Geometry

The **K-Means** algorithm represents the archetype of prototype-based partitional methods. Historically introduced by Hugo Steinhaus in 1956, formalized

by Stuart Lloyd (1957/1982) in signal quantization, and named by James MacQueen (1967), the method produces a hard clustering where every data point is uniquely assigned to exactly one cluster.

Definition 7.2.1 (Hard Partition and Centroid). Given a dataset of n points $\mathcal{D} \subset \mathbb{R}^d$ and a fixed positive integer $K \in \mathbb{N}^+$, a hard partition is a family of subsets $\{C_1, \dots, C_K\}$ such that:

$$\bigcup_{i=1}^K C_i = \mathcal{D}, \quad C_i \cap C_j = \emptyset \quad (\forall i \neq j), \quad C_i \neq \emptyset \quad (\forall i \in \{1, \dots, K\}) \quad (7.1)$$

Each cluster C_i is associated with a prototype vector termed the **centroid** $\mathbf{m}_i \in \mathbb{R}^d$, which for Euclidean distance corresponds to the arithmetic sample mean of the assigned points:

$$\mathbf{m}_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x} \quad (7.2)$$

where $|C_i|$ denotes the cardinality of cluster C_i .

7.2.1 Pseudocode of Lloyd's Algorithm

The standard algorithm alternates recursively between two deterministic phases until convergence, as formulated in Algorithm 1.

7.2.2 Geometric Interpretation: Voronoi Diagram

The assignment step partitions continuous Euclidean space $\mathbb{R}^d$ into K convex polyhedral regions bounded by hyperplanes, known as the **Voronoi tessellation** generated by the prototypes $\{\mathbf{m}_1, \dots, \mathbf{m}_K\}$:

$$V(C_i) = \left\{ \mathbf{x} \in \mathbb{R}^d : \|\mathbf{x} - \mathbf{m}_i\|_2 \leq \|\mathbf{x} - \mathbf{m}_j\|_2 \quad \forall j \neq i \right\} \quad (7.3)$$

Each separating hyperplane between cluster C_i and cluster C_j is orthogonal to the line segment connecting $\mathbf{m}_i$ and $\mathbf{m}_j$ and passes through their midpoint. Consequently, every cluster identified by K-Means is a convex polyhedron; data lying on non-convex or manifold geometries cannot be captured without fracturing.

7.2.3 Temporal Dynamics Across Iterations

Analyzing centroid trajectories across iterations $t = 1, 2, \dots$:

Algorithm 1: Standard K-Means algorithm (Lloyd).

Input: Dataset $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^d$, number of clusters K , tolerance threshold $\epsilon \geq 0$.

Output: Final partition $\{C_1, \dots, C_K\}$ and centroids $\{\mathbf{m}_1, \dots, \mathbf{m}_K\}$.

```
1 Initialize  $K$  centroids  $\{\mathbf{m}_1^{(0)}, \dots, \mathbf{m}_K^{(0)}\}$  via random uniform selection from  $\mathcal{D}$ 
2  $t \leftarrow 0$ 
3 repeat
4    $t \leftarrow t + 1$ 
5   // Phase 1: Assignment to Voronoi cells
6   Initialize  $C_i^{(t)} \leftarrow \emptyset$  for each  $i \in \{1, \dots, K\}$ 
7   foreach  $\mathbf{x} \in \mathcal{D}$  do
8     Determine index of closest centroid:
9      $k^* = \arg \min_{j \in \{1, \dots, K\}} \|\mathbf{x} - \mathbf{m}_j^{(t-1)}\|^2$ 
10    Assign  $\mathbf{x}$  to corresponding cluster:  $C_{k^*}^{(t)} \leftarrow C_{k^*}^{(t)} \cup \{\mathbf{x}\}$ 
11  end
12  // Phase 2: Prototype recomputation (sample means)
13  foreach  $i \in \{1, \dots, K\}$  do
14    if  $|C_i^{(t)}| > 0$  then
15       $\mathbf{m}_i^{(t)} \leftarrow \frac{1}{|C_i^{(t)}|} \sum_{\mathbf{x} \in C_i^{(t)}} \mathbf{x}$ 
16    end
17    else
18      Handle empty cluster by reallocating an orphan centroid
19    end
20  end
21 until centroids do not change significantly:  $\sum_{i=1}^K \|\mathbf{m}_i^{(t+1)} - \mathbf{m}_i^{(t)}\|^2 \leq \epsilon$ 
```

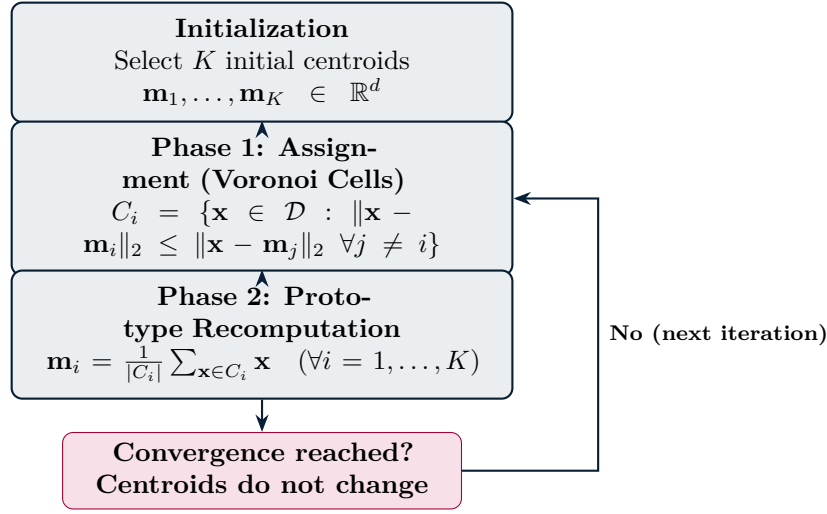


Figure 7.2: Two-phase iterative update cycle of the K-Means algorithm.

- **Iterations 1–3:** Large macroscopic migration occurs. Initial centroids rapidly escape spurious positions and move toward the centers of high-density regions.
- **Iterations 4–6 and onward:** Fine-tuning phase. Only a few boundary points swap across Voronoi boundaries until centroid shifts fall below the tolerance ϵ .

7.2.4 Generalization to Other Proximity Measures

While K-Means is traditionally formulated for Euclidean distance, the two-phase framework generalizes to other metrics provided the prototype is appropriately chosen:

- **Euclidean Distance (L_2):** Prototype = **Mean** (centroid). Minimizes the sum of squared errors (SSE).
- **Manhattan Distance (L_1):** Prototype = Component-wise **Median** (**K-Medians** algorithm). Minimizes the sum of absolute errors (SAE = $\sum_i \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|_1$).
- **Cosine Similarity:** Prototype = **Normalized Mean** to unit norm ($\mathbf{m}_i / \|\mathbf{m}_i\|_2$). Maximizes cumulative cosine similarity (widely used in text mining).
- **Pearson Correlation:** Prototype = **Standardized Vector** (zero mean, unit variance).

7.2.5 Computational Complexity Analysis

Lloyd’s algorithm exhibits exceptional computational efficiency:

- **Time Complexity:** In each iteration, assigning n points to K centroids in d dimensions requires $\mathcal{O}(n \cdot K \cdot d)$ operations. Recomputing centroids takes $\mathcal{O}(n \cdot d)$. Denoting by I the number of iterations to convergence, total time complexity is:

$$\mathcal{T}_{\text{K-Means}} = \mathcal{O}(n \cdot K \cdot I \cdot d) \quad (7.4)$$

Since empirically $K \ll n$, $d \ll n$, and $I \ll n$ (often $I \leq 50$), time scales linearly with dataset size n .

- **Space Complexity:** Storing the data matrix $n \times d$, K centroid vectors of dimension d , and the cluster assignment vector $n \times 1$ requires:

$$\mathcal{S}_{\text{K-Means}} = \mathcal{O}((n + K) \cdot d) \quad (7.5)$$

making the algorithm readily scalable to massive datasets residing in main memory.

7.3 SSE Objective Function and Proof of Convergence

The analytical stability of K-Means stems from the presence of a well-defined scalar objective function that is strictly minimized by the alternating steps of Lloyd’s method.

Definition 7.3.1 (Sum of Squared Errors). For a partition $\{C_1, \dots, C_K\}$ with prototypes $\{\mathbf{m}_1, \dots, \mathbf{m}_K\}$, the **sum of squared errors** (SSE, or within-cluster inertia) is defined as:

$$\text{SSE} = \sum_{i=1}^K \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|_2^2 = \sum_{i=1}^K \sum_{\mathbf{x} \in C_i} \sum_{j=1}^d (x_j - m_{ij})^2 \quad (7.6)$$

7.3.1 Proof: The Sample Mean Strictly Minimizes SSE

Theorem 7.3.1 (Optimality of the Centroid Mean). *Given a cluster C_k containing $|C_k|$ points in $\mathbb{R}^d$, the prototype $\mathbf{m}_k \in \mathbb{R}^d$ that minimizes the sum of squared Euclidean distances to all points in C_k is uniquely the arithmetic sample mean of those points.*

Proof. The squared error for a single cluster C_k decomposes independently across the d orthogonal dimensions:

$$\text{SSE}_k = \sum_{\mathbf{x} \in C_k} \sum_{j=1}^d (x_j - m_{kj})^2 = \sum_{j=1}^d \left(\sum_{\mathbf{x} \in C_k} (x_j - m_{kj})^2 \right) \quad (7.7)$$

To find the minimum with respect to each scalar coordinate m_{kj} of centroid $\mathbf{m}_k$, we compute the first partial derivative and set it to zero:

$$\begin{aligned} \frac{\partial \text{SSE}_k}{\partial m_{kj}} &= \frac{\partial}{\partial m_{kj}} \sum_{\mathbf{x} \in C_k} (x_j - m_{kj})^2 \\ &= \sum_{\mathbf{x} \in C_k} 2(x_j - m_{kj}) \cdot (-1) \\ &= -2 \sum_{\mathbf{x} \in C_k} x_j + 2 \sum_{\mathbf{x} \in C_k} m_{kj} = 0 \end{aligned} \quad (7.8)$$

Since m_{kj} is constant with respect to the summation over $\mathbf{x} \in C_k$, the second sum evaluates to $|C_k| \cdot m_{kj}$:

$$2|C_k| m_{kj} = 2 \sum_{\mathbf{x} \in C_k} x_j \implies m_{kj} = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} x_j \quad (7.9)$$

Verifying the second derivative:

$$\frac{\partial^2 \text{SSE}_k}{\partial m_{kj}^2} = 2|C_k| > 0 \quad (7.10)$$

The corresponding Hessian matrix is a strictly positive-definite scalar diagonal matrix:

$$\nabla^2 \text{SSE}_k = 2|C_k| \mathbf{I}_d \succ 0 \quad (7.11)$$

Therefore, the sample mean $\mathbf{m}_k = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} \mathbf{x}$ is the unique strict global minimizer of SSE_k . $\square$

7.3.2 Coordinate Descent Formulation and Convergence Proof

Lloyd's algorithm can be rigorously formulated as **Coordinate Descent** on a bivariate objective function.

Let $\mathbf{W} \in \{0, 1\}^{n \times K}$ be the binary assignment matrix with $w_{ik} = 1$ if instance $\mathbf{x}_i \in C_k$ and $w_{ik} = 0$ otherwise, subject to $\sum_{k=1}^K w_{ik} = 1$ for all i . Let $\mathbf{S} = (\mathbf{m}_1, \dots, \mathbf{m}_K) \in \mathbb{R}^{d \times K}$ represent the matrix of centroids. The global objective function is:

$$G(\mathbf{W}, \mathbf{S}) = \sum_{i=1}^n \sum_{k=1}^K w_{ik} \|\mathbf{x}_i - \mathbf{m}_k\|_2^2 \quad (7.12)$$

Theorem 7.3.2 (Monotone Convergence to a Local Minimum). *The K-Means algorithm monotonically decreases the objective function $G(\mathbf{W}, \mathbf{S})$ at every half-step and converges in a finite number of iterations to a local minimum or stationary point.*

Proof. Execution alternates between two optimization steps:

1. **Step 1 (Assignment):** Holding centroids $\mathbf{S}^{(t-1)}$ fixed, minimize $G(\mathbf{W}, \mathbf{S}^{(t-1)})$ over $\mathbf{W}$. Because each observation's contribution is decoupled, the objective is minimized by setting:

$$w_{ik}^{(t)} = \begin{cases} 1 & \text{if } k = \arg \min_j \|\mathbf{x}_i - \mathbf{m}_j^{(t-1)}\|_2^2 \\ 0 & \text{otherwise} \end{cases} \quad (7.13)$$

Consequently:

$$G(\mathbf{W}^{(t)}, \mathbf{S}^{(t-1)}) \leq G(\mathbf{W}^{(t-1)}, \mathbf{S}^{(t-1)}) \quad (7.14)$$

2. **Step 2 (Centroid Update):** Holding assignments $\mathbf{W}^{(t)}$ fixed, minimize $G(\mathbf{W}^{(t)}, \mathbf{S})$ over $\mathbf{S}$. By the centroid optimality theorem, the optimal choice for each centroid $\mathbf{m}_k$ is:

$$\mathbf{m}_k^{(t)} = \frac{\sum_{i=1}^n w_{ik}^{(t)} \mathbf{x}_i}{\sum_{i=1}^n w_{ik}^{(t)}} = \frac{1}{|C_k^{(t)}|} \sum_{\mathbf{x} \in C_k^{(t)}} \mathbf{x} \quad (7.15)$$

Consequently:

$$G(\mathbf{W}^{(t)}, \mathbf{S}^{(t)}) \leq G(\mathbf{W}^{(t)}, \mathbf{S}^{(t-1)}) \quad (7.16)$$

Combining both inequalities gives a monotonically non-increasing sequence:

$$G(\mathbf{W}^{(t)}, \mathbf{S}^{(t)}) \leq G(\mathbf{W}^{(t-1)}, \mathbf{S}^{(t-1)}) \quad (7.17)$$

Since $G(\mathbf{W}, \mathbf{S}) \geq 0$ is bounded below by zero, and the set of possible discrete partition matrices $\mathbf{W}$ is finite (given by the Stirling numbers of the second kind $S(n, K)$), the algorithm cannot cycle indefinitely and must reach convergence to a stationary point (local minimum) in a finite number of steps. $\square$

7.4 Choice of Parameter K, Local Minima, and Initialization

Despite guaranteed convergence, the optimization landscape of K-Means is non-convex and laden with sub-optimal local minima. In fact, finding the globally optimal clustering that minimizes SSE is known to be **NP-hard** even for $K = 2$ in general dimensions, or for $d = 2$ with general K .

7.4.1 Determining the Number of Clusters: The Elbow Method

Because $\text{SSE}(K)$ monotonically decreases as K increases (trivially reaching zero at $K = n$), one cannot minimize SSE directly with respect to K .

The standard empirical approach is the **Elbow Method**:

1. Run K-Means across a range of candidate values $K \in \{1, 2, \dots, K_{\max}\}$;
2. Plot the resulting SSE against K ;
3. Identify the inflection point ("elbow") where the marginal reduction in error sharply diminishes.

Analytically, the elbow corresponds to the value of K that maximizes the discrete second derivative of SSE:

$$\Delta^2 \text{SSE}(K) = \text{SSE}(K + 1) - 2\text{SSE}(K) + \text{SSE}(K - 1) \quad (7.18)$$

Beyond this point, introducing further clusters splits natural groups without providing substantial explanatory gain.

7.4.2 Sensitivity to Initial Centroids and Combinatorial Proof

The final clustering quality produced by K-Means depends critically on the initial centroid placement $\mathbf{m}_1^{(0)}, \dots, \mathbf{m}_K^{(0)}$. If two centroids land in the same true cluster, an adjacent natural group is left without a dedicated prototype, trapping the algorithm in an inferior local minimum.

Theorem 7.4.1 (Combinatorial Probability of Pure Random Initialization). *Consider an ideal dataset containing K well-separated natural clusters of equal size N/K . Under uniform random sampling with replacement, the probability $P(K)$ that each selected initial centroid falls into a distinct natural cluster is:*

$$P(K) = \frac{K!}{K^K} \quad (7.19)$$

Proof. We evaluate the probability as a product of conditional probabilities step by step:

1. The first centroid can land in any of the K clusters: $P_1 = \frac{K}{K} = 1$.
2. For the second centroid to land in a different cluster, it must be drawn from one of the remaining $K - 1$ unoccupied clusters. Since all clusters have equal size, $P_2 = \frac{K-1}{K}$.

3. The third centroid must fall into one of the remaining $K - 2$ unoccupied clusters: $P_3 = \frac{K-2}{K}$.
4. Continuing up to the K -th centroid yields $P_K = \frac{1}{K}$.

Multiplying these independent probabilities:

$$P(K) = \prod_{j=1}^K \frac{K-j+1}{K} = \frac{K \cdot (K-1) \cdot (K-2) \cdots 1}{K^K} = \frac{K!}{K^K} \quad (7.20)$$

□

Evaluating equation (7.19) for increasing values of K :

- $K = 2 \implies P(2) = \frac{2}{4} = 0.50$ (50%);
- $K = 3 \implies P(3) = \frac{6}{27} \approx 0.222$ (22.2%);
- $K = 5 \implies P(5) = \frac{120}{3125} \approx 0.0384$ (3.84%);
- $K = 10 \implies P(10) = \frac{3628800}{10^{10}} \approx 0.00036$ (a mere **0.036%**).

Applying Stirling's approximation $K! \approx \sqrt{2\pi K} \left(\frac{K}{e}\right)^K$:

$$P(K) \approx \frac{\sqrt{2\pi K} \left(\frac{K}{e}\right)^K}{K^K} = \sqrt{2\pi K} \cdot e^{-K} \quad (7.21)$$

The probability of successful random initialization decays **exponentially** toward zero as K grows. For $K \geq 10$, naive random initialization almost never lands one centroid per true cluster.

7.5 Initialization Heuristics and Variants of K-Means

To counteract the fragility of naive initialization, robust initialization heuristics and structural variants have been developed.

7.5.1 Heuristic Initialization Strategies

1. **Random Restarts:** Run the algorithm R times (e.g., $R = 50$) with different random seeds, retaining the partition yielding minimum SSE. Effective for small K , but computationally prohibitive and insufficient for large K .

2. **Sampling with Hierarchical Pre-Clustering:** Extract a small random sample ($\approx 1\% - 5\%$) of the data and apply agglomerative hierarchical clustering with Ward's method to identify K stable groups, using their centroids as seeds for full-scale K-Means.
3. **K-Means++ (D^2 -Weighting):** Introduced by Arthur and Vassilvitskii (2007), chooses the first centroid uniformly at random, and each subsequent centroid with probability proportional to the squared distance to the nearest existing centroid:

$$p(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_{\mathbf{y} \in \mathcal{D}} D(\mathbf{y})^2}, \quad D(\mathbf{x}) = \min_{j \in \{1, \dots, m\}} \|\mathbf{x} - \mathbf{m}_j\|_2 \quad (7.22)$$

Theoretical guarantee: K-Means++ guarantees that the expected final SSE is within a logarithmic factor of the global optimum:

$$\mathbb{E}[\text{SSE}] \leq 8(\ln K + 2) \cdot \text{SSE}_{\text{opt}} = \mathcal{O}(\log K) \cdot \text{SSE}_{\text{opt}} \quad (7.23)$$

4. **Bisecting K-Means:** A hybrid divisive approach relying on recursive binary bisections ($K = 2$):
 - Start with the entire dataset treated as a single macro-cluster;
 - At each step, select the cluster with the highest within-cluster SSE (or largest cardinality);
 - Apply standard K-Means with $K = 2$ to split this cluster into two sub-clusters;
 - Repeat until exactly K clusters are formed.

Bisecting K-Means is substantially less sensitive to initialization than Lloyd's method and inherently produces a hierarchical tree of clusters.

7.5.2 Incremental Centroid Update (Online K-Means)

In MacQueen's online formulation (1967), centroids are updated *immediately* after every individual point reassignment, rather than at the end of each epoch:

Property 7.5.1 (MacQueen's Incremental Update Formulas). *When point $\mathbf{x}$ is removed from cluster C_j and added to cluster C_i :*

$$\mathbf{m}_j \leftarrow \mathbf{m}_j - \frac{\mathbf{x} - \mathbf{m}_j}{|C_j| - 1} \quad (7.24)$$

$$\mathbf{m}_i \leftarrow \mathbf{m}_i + \frac{\mathbf{x} - \mathbf{m}_i}{|C_i| + 1} \quad (7.25)$$

This incremental update guarantees that SSE decreases with every individual movement, eliminates empty cluster occurrences, but introduces sensitivity to data presentation order.

7.5.3 Handling Empty Clusters

In batch Lloyd’s algorithm, an assignment phase may leave a centroid with no nearest points ($|C_k| = 0$, an orphan centroid). Standard remediation strategies:

1. **Maximum Residual Selection:** Identify point $\mathbf{x}^* \in \mathcal{D}$ with maximum squared error from its assigned centroid $\|\mathbf{x}^* - \mathbf{m}_{c(\mathbf{x}^*)}\|^2$, and assign it as the new isolated centroid of the empty cluster. This yields the maximum immediate marginal reduction in SSE.
2. **Splitting the Cluster with Maximum SSE:** Find the cluster contributing the highest cumulative SSE and split it by moving its most distant point into the empty cluster.

7.6 Structural Limitations of K-Means and Adaptation Techniques

K-Means implicitly assumes spherical, compact, well-separated clusters of similar volumes and densities. In the presence of complex geometries, the algorithm systematically degrades.

1. Differing Sizes

Centroids divide space evenly; smaller cluster centroid absorbs peripheral points from the larger cluster.

2. Differing Densities

Sparse clusters are fragmented, while compact high-density clusters are improperly merged together.

3. Non-Globular Geometries

Concave or intertwined shapes (*two moons*, concentric rings) are sliced by linear Voronoi boundaries.

4. Outlier Sensitivity

Squared error penalties $\|\mathbf{x} - \mathbf{m}\|^2$ heavily distort centroid locations toward extreme outliers.

Figure 7.3: Four primary structural failure modes of the standard K-Means algorithm.

7.6.1 Analysis of Three Geometric Failure Modes

1. **Differing Sizes:** Because SSE penalizes squared deviations without factoring in intrinsic volume, the centroid of a compact cluster absorbs points from a dispersed cluster to lower overall SSE, distorting cluster boundaries.
2. **Differing Densities:** Equidistant hyperplanes between centroids do not account for density gradients. A sparse cluster requires multiple centroids to cover, forcing adjacent dense clusters to merge into a single group.
3. **Non-Globular Shapes (e.g., Two Moons):** Voronoi cells are convex polyhedra. For intertwined non-convex manifolds (two interlocking crescents), linear hyperplanes cut across both curves, failing entirely to separate them.

Remediation Strategy: Over-Clustering

To model complex manifolds or uneven densities, one runs K-Means with an over-specified number of micro-clusters ($K' \gg K$). A chain of small centroids provides a piecewise linear approximation along the manifold curve. Subsequent graph-based or linkage routines merge these micro-clusters into the desired non-linear clusters.

7.6.2 Pre-processing and Post-processing: The ISO-DATA Paradigm

To mitigate these limitations in practice, pre- and post-processing routines formalized in the classic **ISODATA** algorithm (*Iterative Self-Organizing Data Analysis Technique*) are commonly used:

- **Pre-processing:**
 - Feature normalization or standardization (z-score) to prevent large-scale attributes from dominating Euclidean distance calculations;
 - Outlier detection and filtering prior to centroid initialization.
- **Post-processing (ISODATA Dynamics):**
 - **Split:** If a cluster's standard deviation or SSE exceeds a maximum threshold θ_{split} and its size is sufficient ($|C_i| \geq 2\theta_{\text{min}}$), split it along its coordinate of maximum variance.

- **Merge:** If the Euclidean distance between two centroids $\|\mathbf{m}_i - \mathbf{m}_j\|_2$ falls below θ_{merge} , merge them into a single centroid.
- **Prune:** If $|C_i| < \theta_{\text{min}}$, eliminate the cluster and reassign its members.

7.7 Foundations of Hierarchical Clustering

Hierarchical Clustering overcomes the limitation of flat partitions by generating a continuous nested sequence of clusters.

Definition 7.7.1 (Dendrogram). A **dendrogram** is a rooted binary tree visualization where:

- The **leaves** at level $h = 0$ represent individual data points (singleton clusters);
- The **internal nodes** represent successive pairwise cluster merges;
- The **height** h of each junction equals the distance or dissimilarity at which the two clusters were merged.

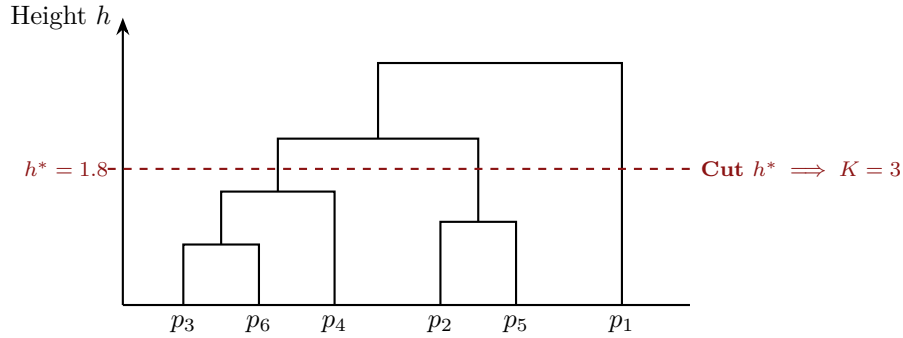


Figure 7.4: Dendrogram structure: junction heights represent merge distances. A horizontal cut yields a flat partition with $K = 3$ clusters.

7.7.1 Distinct Strengths of Hierarchical Clustering

- **No a priori constraint on K :** The user does not need to specify cluster count beforehand. A full visual taxonomy is revealed, and K is chosen post-hoc by drawing a horizontal cut at height h^* .
- **Modeling natural hierarchies:** Biological phylogenies, taxonomies, and linguistic trees possess natural hierarchical structures faithfully captured by dendrograms.

- **Determinism:** Standard hierarchical algorithms are fully deterministic, with no sensitivity to random seeds.

7.7.2 Taxonomy of Hierarchical Algorithms

- **Agglomerative (*Bottom-up*):** Begin with n singleton clusters and iteratively merge the closest pair of clusters in $n - 1$ steps until a single root cluster remains.
- **Divisive (*Top-down*):** Begin with the entire dataset in a single cluster and recursively split clusters into sub-clusters until n singletons are reached.

7.8 Agglomerative Algorithm and Linkage Criteria

The standard agglomerative procedure requires maintaining an inter-cluster proximity matrix $\mathbf{D} \in \mathbb{R}^{N \times N}$, following Algorithm 2.

Algorithm 2: Standard agglomerative hierarchical clustering algorithm.

Input: Dataset $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, distance metric $d(\cdot, \cdot)$.

Output: Complete hierarchical dendrogram of merges.

- 1 Initialize n singleton clusters: $C_i = \{\mathbf{x}_i\}$ for each $i \in \{1, \dots, n\}$
 - 2 Compute initial proximity matrix $\mathbf{D}^{(0)} \in \mathbb{R}^{n \times n}$ where $D_{ij} = d(\mathbf{x}_i, \mathbf{x}_j)$
 - 3 **for** $t = 1$ **to** $n - 1$ **do**
 - 4 Find the closest pair of clusters: $(A, B) = \arg \min_{i \neq j} D_{ij}^{(t-1)}$
 - 5 Merge clusters A and B into $A \cup B$ at height $h_t = D_{AB}^{(t-1)}$
 - 6 Remove rows and columns corresponding to A and B from the matrix
 - 7 Compute distance between new cluster $A \cup B$ and every remaining cluster Q according to chosen linkage criterion
 - 8 Update proximity matrix $\mathbf{D}^{(t)}$ with the new row and column for $A \cup B$
 - 9 **end**
-

7.8.1 The Four Principal Linkage Criteria

The definition of inter-cluster distance $D(A, B)$ governs the geometry and properties of the resulting hierarchy.

1. MIN (Single Link)

Distance between two clusters equals the minimum distance between any pair of constituent points:

$$D_{\text{MIN}}(A, B) = \min_{\mathbf{x} \in A, \mathbf{y} \in B} d(\mathbf{x}, \mathbf{y}) \quad (7.26)$$

- **Strengths:** Capable of detecting arbitrary non-convex, elongated, or concentric shapes (isomorphic to Kruskal's Minimum Spanning Tree construction).
- **Weaknesses (*Chaining Effect*):** Highly susceptible to noise. A single bridge of noise points between two well-separated clusters causes them to prematurely merge into a long chain.

2. MAX (Complete Link)

Distance equals the maximum distance between any pair of constituent points:

$$D_{\text{MAX}}(A, B) = \max_{\mathbf{x} \in A, \mathbf{y} \in B} d(\mathbf{x}, \mathbf{y}) \quad (7.27)$$

- **Strengths:** Immune to chaining; enforces compact, spherical clusters by strictly bounding the cluster diameter: $\text{diam}(A \cup B) \leq h$.
- **Weaknesses:** Tends to fragment elongated clusters and is sensitive to peripheral outliers.

3. Group Average (Average Link / UPGMA)

Distance is defined as the average distance between all pairs of cross-cluster points:

$$D_{\text{AVG}}(A, B) = \frac{1}{|A| \cdot |B|} \sum_{\mathbf{x} \in A} \sum_{\mathbf{y} \in B} d(\mathbf{x}, \mathbf{y}) \quad (7.28)$$

A robust compromise between MIN and MAX. Less sensitive to noise than Single Link because individual outliers are averaged across all pairwise distances.

4. Ward's Method (Variance Minimization)

Introduced by Joe H. Ward Jr. (1963), proximity between two clusters is measured by the **increase in total squared error** (ΔSSE) resulting from their merge:

$$\Delta\text{SSE}_{AB} = \text{SSE}_{A \cup B} - (\text{SSE}_A + \text{SSE}_B) \quad (7.29)$$

Theorem 7.8.1 (Closed-Form Expression for Ward's Method). *The increase in sum of squared errors ΔSSE_{AB} caused by merging clusters A and B equals the squared Euclidean distance between their centroids, scaled by the harmonic product of their cardinalities:*

$$\Delta\text{SSE}_{AB} = \frac{|A| \cdot |B|}{|A| + |B|} \|\mathbf{m}_A - \mathbf{m}_B\|_2^2 \quad (7.30)$$

Proof. Let A and B be disjoint clusters with cardinalities $|A|, |B|$ and centroids $\mathbf{m}_A, \mathbf{m}_B$. The centroid of the merged cluster $A \cup B$ is:

$$\mathbf{m}_{A \cup B} = \frac{|A|\mathbf{m}_A + |B|\mathbf{m}_B}{|A| + |B|} \quad (7.31)$$

The displacement vector between the merged centroid and $\mathbf{m}_A$ is:

$$\mathbf{m}_{A \cup B} - \mathbf{m}_A = \frac{|B|(\mathbf{m}_B - \mathbf{m}_A)}{|A| + |B|} \quad (7.32)$$

Symmetrically for cluster B :

$$\mathbf{m}_{A \cup B} - \mathbf{m}_B = \frac{|A|(\mathbf{m}_A - \mathbf{m}_B)}{|A| + |B|} \quad (7.33)$$

The total squared error of the union is:

$$\text{SSE}_{A \cup B} = \sum_{\mathbf{x} \in A} \|\mathbf{x} - \mathbf{m}_{A \cup B}\|^2 + \sum_{\mathbf{y} \in B} \|\mathbf{y} - \mathbf{m}_{A \cup B}\|^2 \quad (7.34)$$

Expanding the first summation by inserting and subtracting $\mathbf{m}_A$:

$$\begin{aligned} \sum_{\mathbf{x} \in A} \|\mathbf{x} - \mathbf{m}_{A \cup B}\|^2 &= \sum_{\mathbf{x} \in A} \|(\mathbf{x} - \mathbf{m}_A) - (\mathbf{m}_{A \cup B} - \mathbf{m}_A)\|^2 \\ &= \sum_{\mathbf{x} \in A} \|\mathbf{x} - \mathbf{m}_A\|^2 + \sum_{\mathbf{x} \in A} \|\mathbf{m}_{A \cup B} - \mathbf{m}_A\|^2 \\ &\quad - 2 \left(\sum_{\mathbf{x} \in A} (\mathbf{x} - \mathbf{m}_A) \right) \cdot (\mathbf{m}_{A \cup B} - \mathbf{m}_A) \end{aligned} \quad (7.35)$$

Since $\sum_{\mathbf{x} \in A} (\mathbf{x} - \mathbf{m}_A) = \mathbf{0}$ by definition of the centroid, the cross-term vanishes:

$$\sum_{\mathbf{x} \in A} \|\mathbf{x} - \mathbf{m}_{A \cup B}\|^2 = \text{SSE}_A + |A| \cdot \|\mathbf{m}_{A \cup B} - \mathbf{m}_A\|^2 \quad (7.36)$$

Substituting the displacement expression:

$$\begin{aligned} \sum_{\mathbf{x} \in A} \|\mathbf{x} - \mathbf{m}_{A \cup B}\|^2 &= \text{SSE}_A + |A| \left\| \frac{|B|(\mathbf{m}_B - \mathbf{m}_A)}{|A| + |B|} \right\|^2 \\ &= \text{SSE}_A + \frac{|A| \cdot |B|^2}{(|A| + |B|)^2} \|\mathbf{m}_A - \mathbf{m}_B\|^2 \end{aligned} \quad (7.37)$$

Repeating for cluster B :

$$\sum_{\mathbf{y} \in B} \|\mathbf{y} - \mathbf{m}_{A \cup B}\|^2 = \text{SSE}_B + \frac{|A|^2 \cdot |B|}{(|A| + |B|)^2} \|\mathbf{m}_A - \mathbf{m}_B\|^2 \quad (7.38)$$

Summing both contributions yields $\text{SSE}_{A \cup B}$:

$$\begin{aligned} \text{SSE}_{A \cup B} &= \text{SSE}_A + \text{SSE}_B + \left[\frac{|A| \cdot |B|^2 + |A|^2 \cdot |B|}{(|A| + |B|)^2} \right] \|\mathbf{m}_A - \mathbf{m}_B\|^2 \\ &= \text{SSE}_A + \text{SSE}_B + \left[\frac{|A| \cdot |B|(|B| + |A|)}{(|A| + |B|)^2} \right] \|\mathbf{m}_A - \mathbf{m}_B\|^2 \\ &= \text{SSE}_A + \text{SSE}_B + \frac{|A| \cdot |B|}{|A| + |B|} \|\mathbf{m}_A - \mathbf{m}_B\|^2 \end{aligned} \quad (7.39)$$

Subtracting $(\text{SSE}_A + \text{SSE}_B)$ proves the claim:

$$\Delta \text{SSE}_{AB} = \text{SSE}_{A \cup B} - (\text{SSE}_A + \text{SSE}_B) = \frac{|A| \cdot |B|}{|A| + |B|} \|\mathbf{m}_A - \mathbf{m}_B\|^2 \quad (7.40)$$

□

Ward's method serves as the exact hierarchical analogue of the K-Means objective function: it produces compact, spherical, balanced clusters and exhibits strong noise tolerance.

7.8.2 Worked Numerical Example on 6 Points: MIN vs MAX

Consider a dataset of 6 points $\{p_1, \dots, p_6\}$ with initial distance matrix:

$$\mathbf{D} = \begin{pmatrix} & p_1 & p_2 & p_3 & p_4 & p_5 & p_6 \\ p_1 & 0.00 & 0.24 & 0.22 & 0.37 & 0.34 & 0.23 \\ p_2 & 0.24 & 0.00 & 0.15 & 0.20 & 0.14 & 0.25 \\ p_3 & 0.22 & 0.15 & 0.00 & 0.15 & 0.28 & 0.11 \\ p_4 & 0.37 & 0.20 & 0.15 & 0.00 & 0.29 & 0.22 \\ p_5 & 0.34 & 0.14 & 0.28 & 0.29 & 0.00 & 0.39 \\ p_6 & 0.23 & 0.25 & 0.11 & 0.22 & 0.39 & 0.00 \end{pmatrix} \quad (7.41)$$

Resolution with MIN (Single Link)

1. Global minimum: $d(p_3, p_6) = 0.11 \implies$ Merge $\{p_3, p_6\}$ at $h = 0.11$.
2. Next minimum: $d(p_2, p_5) = 0.14 \implies$ Merge $\{p_2, p_5\}$ at $h = 0.14$.
3. Distance from $\{p_3, p_6\}$ to p_4 : $\min(d(p_3, p_4), d(p_6, p_4)) = 0.15 \implies$ Merge into $\{p_3, p_6, p_4\}$ at $h = 0.15$.
4. Distance between $\{p_3, p_6, p_4\}$ and $\{p_2, p_5\}$:

$$\begin{aligned} \min_{\mathbf{x} \in \{3,6,4\}, \mathbf{y} \in \{2,5\}} d(\mathbf{x}, \mathbf{y}) &= \min(0.15, 0.25, 0.20, 0.28, 0.39, 0.29) \\ &= d(p_3, p_2) = 0.15 \end{aligned} \quad (7.42)$$

The two blocks merge immediately at height $h = 0.15$ via link p_3 – p_2 .

5. Distance to p_1 : $\min(d(p_1, p_j)) = d(p_1, p_3) = 0.22 \implies$ Final merge at $h = 0.22$.

Resolution with MAX (Complete Link)

1. First two steps match MIN: merge $\{p_3, p_6\}$ at $h = 0.11$, and $\{p_2, p_5\}$ at $h = 0.14$.
2. MAX distance from $\{p_3, p_6\}$ to p_4 : $\max(d(p_3, p_4), d(p_6, p_4)) = 0.22$.
3. MAX distance between $\{p_3, p_6\}$ and $\{p_2, p_5\}$:

$$\begin{aligned} \max_{\mathbf{x} \in \{3,6\}, \mathbf{y} \in \{2,5\}} d(\mathbf{x}, \mathbf{y}) &= \max(0.15, 0.25, 0.28, 0.39) \\ &= d(p_6, p_5) = 0.39 \end{aligned} \quad (7.43)$$

Since $0.22 < 0.39$, the preferred merge is $\{p_3, p_6\}$ with p_4 at $h = 0.22$.

4. Distance between $\{p_3, p_6, p_4\}$ and $\{p_2, p_5\}$ remains high (0.39). In contrast, distance from p_1 to $\{p_2, p_5\}$ is $\max(0.24, 0.34) = 0.34$. Thus, MAX postpones merging the two large groups, avoiding premature linkage chains.

7.8.3 Lance-Williams Recurrence Formula

The recurrence formula by Lance and Williams (1967) enables updating proximity matrices algebraically without recomputing individual point distances.

Theorem 7.8.2 (Lance-Williams Formula). *When clusters A and B merge into $A \cup B$, the distance to any remaining cluster Q satisfies:*

$$D(A \cup B, Q) = \alpha_A D(A, Q) + \alpha_B D(B, Q) + \beta D(A, B) + \gamma |D(A, Q) - D(B, Q)| \quad (7.44)$$

where parameters $(\alpha_A, \alpha_B, \beta, \gamma)$ define the specific linkage metric.

Linkage Criterion	α_A	α_B	β	γ
MIN (Single Link)	1/2	1/2	0	-1/2
MAX (Complete Link)	1/2	1/2	0	+1/2
Group Average	$\frac{ A }{ A + B }$	$\frac{ B }{ A + B }$	0	0
Centroid Linkage	$\frac{ A }{ A + B }$	$\frac{ B }{ A + B }$	$-\frac{ A B }{(A + B)^2}$	0
Ward's Method	$\frac{ A + Q }{ A + B + Q }$	$\frac{ B + Q }{ A + B + Q }$	$-\frac{ Q }{ A + B + Q }$	0

Table 7.1: Lance-Williams recurrence coefficients for major hierarchical methods.

Consistency Proof for Group Average. Substituting the Lance-Williams coefficients for Group Average:

$$\begin{aligned} \sum_{\mathbf{x} \in A \cup B} \sum_{\mathbf{z} \in Q} d(\mathbf{x}, \mathbf{z}) &= \sum_{\mathbf{x} \in A} \sum_{\mathbf{z} \in Q} d(\mathbf{x}, \mathbf{z}) + \sum_{\mathbf{y} \in B} \sum_{\mathbf{z} \in Q} d(\mathbf{y}, \mathbf{z}) \\ &= |A| \cdot |Q| \cdot D_{\text{AVG}}(A, Q) + |B| \cdot |Q| \cdot D_{\text{AVG}}(B, Q) \end{aligned} \quad (7.45)$$

Dividing by $|A \cup B| \cdot |Q| = (|A| + |B|) \cdot |Q|$ and simplifying $|Q| > 0$:

$$D_{\text{AVG}}(A \cup B, Q) = \frac{|A|}{|A| + |B|} D_{\text{AVG}}(A, Q) + \frac{|B|}{|A| + |B|} D_{\text{AVG}}(B, Q) \quad (7.46)$$

matching the coefficients in Table 7.1. $\square$

7.8.4 Synoptic Comparison of Linkage Criteria

Criterion	Objective	Geometry	Outliers	Properties
MIN (Single)	$\min_{\mathbf{x}, \mathbf{y}} d(\mathbf{x}, \mathbf{y})$	Arbitrary, chaining	Very high (<i>chaining</i>)	Isomorphic to MST
MAX (Complete)	$\max_{\mathbf{x}, \mathbf{y}} d(\mathbf{x}, \mathbf{y})$	Spherical, compact	Low at core	Diameter $\leq h$
Group Average	$\frac{\sum \sum d}{ A B }$ Pairwise	Globular tolerant	Moderate	Balanced
Ward's Method	ΔSSE_{AB}	Spherical balanced	Very low	K-Means analogue

Table 7.2: Analytical comparison across the four principal linkage criteria in hierarchical clustering.

7.9 MST-Based Divisive Hierarchical Clustering

While naive divisive clustering is computationally intractable (evaluating all $2^{n-1} - 1$ possible binary splits), graph theory offers an elegant and polynomial formulation using the **Minimum Spanning Tree** (MST).

Definition 7.9.1 (Minimum Spanning Tree of Data Points). Given complete graph $G = (V, E)$ where vertices V represent the n data points and edge weights are $w(u, v) = d(\mathbf{x}_u, \mathbf{x}_v)$, an MST is a cycle-free subgraph connecting all vertices with minimum total weight:

$$w(\text{MST}) = \sum_{e \in \text{MST}} w(e) \rightarrow \min \quad (7.47)$$

7.9.1 Divisive Algorithm via Edge Deletion

Property 7.9.1 (Equivalence with Single Link Agglomerative Clustering). *The partition produced by removing the $K - 1$ heaviest edges from an MST is **strictly identical** to the partition obtained by running Single Link agglomerative clustering down to K clusters. Consequently, Prim's algorithm ($\mathcal{O}(n^2)$) provides an efficient direct path to construct the Single Link hierarchy.*

Algorithm 3: Divisive clustering based on Minimum Spanning Tree.

Input: Dataset $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, desired number of clusters K .

Output: Partition $\{C_1, \dots, C_K\}$.

- 1 Construct complete weighted graph $G = (V, E)$ from pairwise distances
 - 2 Extract Minimum Spanning Tree $\mathcal{T}_{\text{MST}}$ using Prim's or Kruskal's algorithm
 - 3 Sort the $n - 1$ tree edges in non-increasing order:
 $w(e_1) \geq w(e_2) \geq \dots \geq w(e_{n-1})$
 - 4 **for** $k = 1$ **to** $K - 1$ **do**
 - 5 Delete longest remaining edge e_k from the spanning tree
 - 6 The tree splits into two disjoint connected components
 - 7 **end**
 - 8 Return the resulting K connected components as the final clusters
-

7.10 Complexity, Structural Limits, and Synoptic Comparison

7.10.1 Asymptotic Complexity Analysis

- **Space Complexity:** Hierarchical clustering requires storing the full $n \times n$ proximity matrix:

$$\mathcal{S}_{\text{Hierarchical}} = \mathcal{O}(n^2) \quad (7.48)$$

This presents a formidable bottleneck in Big Data scenarios. For $n = 100\,000$ points with double precision (8 bytes), the matrix requires approximately 40 GB of RAM.

- **Naive Time Complexity:** With $n - 1$ merge iterations and linear search across an $\mathcal{O}(n^2)$ matrix at each step:

$$\mathcal{T}_{\text{Naive}} = \sum_{t=1}^{n-1} \mathcal{O}((n-t)^2) = \mathcal{O}(n^3) \quad (7.49)$$

- **Priority Queue (Heap) Optimization:** Maintaining distances in min-heaps reduces merge search and updates to $\mathcal{O}(n \log n)$ per step, lowering overall complexity to:

$$\mathcal{T}_{\text{Heap}} = \mathcal{O}(n^2 \log n) \quad (7.50)$$

(For specific criteria such as Single Link and Complete Link, specialized algorithms like SLINK and CLINK achieve $\mathcal{O}(n^2)$ time with $\mathcal{O}(n)$ auxiliary space).

7.10.2 Structural Limitations of Hierarchical Clustering

1. **Irreversibility of Greedy Decisions:** Merges and splits are permanent without backtracking. If two points merge prematurely due to local noise, they remain tied at all subsequent levels.
2. **Absence of a Global Objective Function:** Except for Ward's method (which minimizes local variance growth), linkage metrics do not optimize a global mathematical cost function across the entire dataset.

7.10.3 Final Synoptic Schema: K-Means vs Hierarchical Clustering

Table 7.3 contrasts the architectural, algebraic, and operational properties of the two paradigms.

Operational Feature	K-Means Clustering	Hierarchical Clustering
Structure Type	Flat partition (<i>hard clustering</i>)	Nested taxonomic tree (Dendrogram)
Parameter K	Mandatory a priori	Derivable post-hoc by drawing a cut line
Objective Function	Global SSE minimization	Greedy local decisions without global cost
Time Complexity	$\mathcal{O}(n \cdot K \cdot I \cdot d)$ (Linear in n)	$\mathcal{O}(n^2 \log n)$ to $\mathcal{O}(n^3)$ (Super-quadratic)
Space Complexity	$\mathcal{O}((n + K) \cdot d)$ (Linear in n)	$\mathcal{O}(n^2)$ (Quadratic memory bottleneck)
Data Scalability	Very high (millions of records)	Low ($n \leq 20\,000$ in standard RAM)
Initialization Sensitivity	Severe (susceptible to local minima)	Fully deterministic (no random seed)
Shape Flexibility	Spherical/globular only (Voronoi hyperplanes)	Arbitrary with Single Link; spherical with Ward
Decision Reversibility	Yes (points reassign at each iteration)	No (permanent merges/splits without backtracking)

Table 7.3: Global synoptic comparison between K-Means and Hierarchical Clustering.

Bibliography